

CURSO SUPERIOR DE TECNOLOGIA EM MANUFATURA AVANÇADA

**RELATÓRIO FINAL DE DESENVOLVIMENTO INTEGRADO DE PRODUTO
“M.E.D LOCK”**

**ELISÂNGELA FERRAZ DE CAMPOS
ERIC DOMINGUES GODOI
HELOISA CLARA RODRIGUES
JOSÉ EDUARDO TEIXEIRA DOS REIS
KAIKE ELIAS FERREIRA DOS ANJOS**

**Profº Orientador: Dawilmar Guimarães de Araújo
Profº Orientador: Alfred Makoto Kabayama
Profª Orientadora: Viviane Ribeiro de Siqueira**

**São José dos Campos
2026**

RESUMO

O projeto emprega a gamificação para tornar processos de aprendizagem e treinamento mais interativos e atrativos, promovendo engajamento, raciocínio lógico e trabalho em equipe. Inspirado no conceito de escape room, o objetivo foi criar uma caixa tipo cofre com um sistema de trava em íris, que é totalmente desbloqueada após a conclusão de três jogos independentes: um eletrônico, um digital e um mecânico. Cada jogo, ao ser finalizado, envia um sinal via comunicação em rede para abertura da íris por completo.

A estrutura do sistema é composta pela Caixa Principal, que contém a trava íris controlada por um microcontrolador ESP32, e pelos três jogos: o Jogo 1 Eletrônico (um “memorização” com LEDs e botões), o Jogo 2 Digital (um aplicativo desenvolvido no MIT App Inventor para celular) e o Jogo 3 Mecânico (um labirinto físico com sensor IR). A integração e funcionamento ocorrem via comunicação sem fio; cada jogo, ao ser concluído, envia um sinal ao ESP32, que abre uma parte da trava. A ordem de conclusão dos jogos é irrelevante, e a íris abre-se por completo apenas quando todos os desafios forem resolvidos.

LISTA DE FIGURAS

Figura 1 - Estrutura da maleta em mdf.....	6
Figura 2 - Painel da Íris montado com as engrenagens, display, leds e interruptores.....	7
Figura 3 - Montagem inicial do painel da Íris.....	8
Figura 4 - Parte posterior do painel e distribuição dos componentes na parte inferior da maleta.....	9
Figura 5 - Tampa que foi inserida para proteger os os componentes e resultado final.....	9
Figura 6 - Circuito Elétrico para o funcionamento do projeto.....	10
Figura 7 - Circuito dos componentes.....	11
Figura 8 - Fluxograma do funcionamento do sistema.....	12
Figura 9 - Diagrama.....	13
Figura 10 - Projeto da maleta e painel finalizado.....	19
Figura 11 - Versão Final.....	20
Figura 12 - Versão Final.....	21
Figura 13 - Diagrama Elétrico.....	22
Figura 14 - Eletrônica Atual.....	23
Figura 15 - Bibliotecas.....	24
Figura 16 - Bibliotecas.....	25
Figura 17 - Fluxograma funcional – Jogo 2 - Digital.....	28
Figura 18 - Tela do Celular – Tela de Jogo - Jogo Digital.....	29
Figura 19 - Tela do Celular – Tela Inicial - Jogo Digital.....	30
Figura 20 - Tela do Tablet – Tela Inicial - Jogo Digital.....	31
Figura 21 - Tela do Tablet V1– Tela de Jogo - Jogo Digital.....	32
Figura 22 - Hardware utilizado no teste de comunicação - Jogo Digital.....	33
Figura 23 - Tela do Tablet V2 – Tela de Jogo - Jogo Digital.....	34
Figura 24 - Tela da aplicação - Jogo Digital.....	36
Figura 25 - Programação em blocos da aplicação - Jogo Digital.....	37
Figura 26 - Testes finais de comunicação - Jogo Digital.....	38
Figura 27 - Suporte para o tablet.....	39
Figura 28 - Código Base do Servidor (ESP32 da Maleta).....	42
Figura 29 - Código Base Cliente (Usado nos Jogos Memória e Labirinto).....	44
Figura 30 - Esquemático do Circuito.....	49
Figura 31 - Diagrama Elétrico do Projeto.....	50
Figura 32 - Primeiro Layout.....	51
Figura 33 - Circuito da Primeiro Modelo em papelão.....	52
Figura 34 - Circuito da Segundo Modelo em papelão.....	53
Figura 35 - Vista explodida do desenho CAD.....	54
Figura 36 - Circuito do Modelo Impresso 3D.....	55
Figura 37 - Modelo 3D Impresso e Montado.....	55

LISTA DE TABELAS

Tabela 1 - Custos totais.....	13
Tabela 2 - Apresenta um resumo dos principais testes realizados.....	17
Tabela 3 - Período de testes de autonomia e recarga.....	26
Tabela 4 - Materiais e custos.....	26
Tabela 2.4A- Custo Total do Projeto.....	46
Tabela 2.4B- Contextualização.....	50

SUMÁRIO

1. INTRODUÇÃO.....	4
2. DESENVOLVIMENTO.....	5
2.1 Maleta e Pannel da Íris Mecânica.....	6
2.1.1 Construção do Pannel.....	6
2.1.2 Sistema da Íris Mecânica.....	7
2.1.3 Sistema Eletrônico.....	9
2.1.4 Programação e Comunicação.....	11
2.1.5 Componentes e tabela de valores.....	13
2.1.7 Sistema de Alimentação.....	14
2.1.8 Testes Realizados.....	15
2.1.9 Desempenho Mecânico.....	16
2.1.10 Desempenho Eletrônico.....	16
2.1.11 Comunicação do Sistema.....	16
2.1.12 Avaliação Estrutural.....	16
2.1.13 Resultados Obtidos.....	17
2.1.14 CONCLUSÃO.....	18
2.2 Jogo 1 – Eletrônico.....	20
2.2.1 Concepção Inicial.....	20
2.2.2 Diagnóstico da Primeira Versão.....	20
2.2.3 Evolução Estrutural.....	21
2.2.4 Evolução Eletrônica.....	22
2.2.5 Integração com a Maleta.....	23
2.2.6 Testes de autonomia.....	25
2.2.7 Custos.....	26
2.2.8 Resultados.....	27
2.2.9 Conclusão.....	27
2.3 Jogo 2 – Digital.....	28
2.4 - Jogo 3 – Eletromecânico: Integração do Projeto e Jogo Labirinto.....	40
2.4.1 - Integração do Projeto.....	40
2.4.2 - Pesquisa e Desenvolvimento.....	40
2.4.3 - Redirecionamento da Solução e Primeiros Testes.....	40
2.4.4 - Ajuste de Arquitetura e Independência de Rede.....	41
2.4.5 - Servidor / Cliente: Códigos Base do Sistema.....	41
2.4.5.1 - Análise do Código do Servidor.....	43
2.4.5.2 - Análise do Código Cliente.....	44
2.4.6 - Integrando cada Jogo à Maleta.....	45
2.4.7 - Observações.....	45
2.4.8 - Conclusão.....	45
2.4.9 - Jogo Labirinto: Objetivo do Projeto.....	46
2.4.10 - Funcionamento.....	46
2.4.11 - Circuito e Operação (Lista de Componentes).....	46
2.4.12 - Bateria (Tempo Estimado de Duração e Carregamento).....	48
2.4.13 - Esquemático.....	48
2.4.14 - Diagrama Elétrico.....	50

2.4.15 - Estrutura (Contextualização do Antigo para o Novo).....	50
2.4.16 - Desenvolvimento do Invólucro em 3D.....	53
2.4.17 - Conclusão.....	56
3. CONCLUSÃO.....	57
REFERÊNCIAS.....	58
APÊNDICE A - Programação completa do Painel da Íris.....	59

1. INTRODUÇÃO

O projeto utiliza gamificação para tornar processos de aprendizado e treinamento mais interativos e atrativos, promovendo engajamento, raciocínio lógico e trabalho em equipe.

O projeto foi inspirado no jogo escape room com objetivo de criar uma caixa tipo cofre com sistema de travamento em três estágios, que é totalmente desbloqueada após a conclusão de três jogos independentes (eletrônico, digital e mecânico). Cada jogo, ao ser finalizado, envia um sinal via rede wifi para abrir uma parte da íris.

A Estrutura do Sistema é composta por:

- **Caixa Principal:**

Contém uma íris mecânica controlada por um microcontrolador esp32.

- **Jogo 1 – Eletrônico (genius):**

Jogo de memória com leds e botões, baseado no clássico “genius”.

- **Jogo 2 – Digital:**

Aplicativo desenvolvido em MIT App Inventor para celular, com desafio lógico.

- **Jogo 3 – Mecânico (labirinto):**

Labirinto físico com sensor IR que detecta quando a bolinha atinge o destino.

Integração e funcionamento: Cada jogo comunica-se sem fio (wifi) com a caixa. Ao concluir um jogo, um sinal é enviado ao ESP32, que abre uma parte da íris. A ordem de conclusão dos jogos é irrelevante.

O cronograma tem o período: 28/08/2025 a 24/06/2026 (apresentação na Feira de Soluções da Fatec).

O projeto foi dividido em: Caixa/íris mecânica: Heloisa Rodrigues; Jogo 1 (Memória): José Eduardo e Eric Godoi; Jogo 2 (Digital): Kaike Anjos; Jogo 3 (Labirinto): Elisângela Campos.

Documentação e integração: Participação coletiva.

Algumas das tecnologias utilizadas: Controle: ESP32 (inicialmente testado com MicroPython, mas implementado em C++), Comunicação: wi-fi.

2. DESENVOLVIMENTO

Este documento detalha as etapas de montagem, a relação de componentes (elétricos e mecânicos) e o desenvolvimento da programação do sistema MED Lock.

O projeto é estruturado em um módulo central, a Caixa, e três desafios independentes, os Jogos de 1 a 3. A abertura da caixa só ocorre quando o usuário resolve cada um dos jogos, que liberam, individualmente, as travas do mecanismo de fechadura.

A seguir, descreveremos integralmente o processo de construção, integração e funcionamento do sistema.

2.1 Maleta e Painel da Íris Mecânica

A estrutura do projeto final foi desenvolvida utilizando uma maleta de mdf de 6mm, no tamanho de 300mm largura x 200mm altura x 130mm profundidade, sendo separado em duas parte superior de 30mm e inferior de 100mm, pois tivemos que trocar, a nova maleta apresenta dimensões adequadas e de fácil adaptação. Além de servir como suporte para os componentes, a maleta proporciona proteção mecânica, organização interna e portabilidade ao equipamento.

O espaço interno foi dividido em dois compartimentos: o primeiro destinado ao painel principal, onde estão instalados a íris mecânica e os elementos de interação, e o segundo reservado para armazenamento dos jogos e organização dos componentes auxiliares.

As peças do painel e da maleta receberam lixamento, pintura interna e externa, melhorando o acabamento, reduzindo o atrito entre os componentes e facilitando futuras manutenções.

Figura 1 - Estrutura da maleta em mdf



Fonte: autoria própria

2.1.1 Construção do Painel

O painel principal foi desenvolvido para integrar os sistemas mecânico e eletrônico do projeto. Sua função é sustentar a íris mecânica, o conjunto de transmissão e os componentes eletrônicos responsáveis pelo funcionamento do equipamento. Além

disso, o painel concentra os elementos de interação com o usuário, como display, LEDs indicadores e interruptores.

Sua estrutura foi confeccionada em MDF de 6 mm, no tamanho de 290mm de largura x 190mm, proporcionando resistência adequada e facilidade de fabricação. A disposição dos componentes foi planejada para aproveitar o espaço disponível, facilitar a montagem e reduzir interferências entre os sistemas.

Na parte frontal foi instalada a íris mecânica juntamente com o display digital e os LEDs indicadores. Na parte traseira foram fixados o servomotor SG90, o ESP32, o regulador LM2596 e os demais componentes eletrônicos. A organização do cabeamento foi realizada de forma a facilitar a manutenção e reduzir o risco de falhas durante o funcionamento.

O circuito eletrônico foi inicialmente desenvolvido no software EasyEDA para validação das conexões elétricas e posteriormente representado no Fritzing, servindo como referência durante a montagem física do protótipo.

Figura 2 - Painel da Íris montado com as engrenagens, display, leds e interruptores.



Fonte: autoria própria (18/05/2026)

2.1.2 Sistema da Íris Mecânica

A íris mecânica representa o principal elemento de interação do projeto, funcionando como mecanismo de desbloqueio progressivo conforme o usuário conclui as etapas dos jogos. Inspirado nas íris utilizadas em câmeras fotográficas, o sistema foi adaptado para oferecer uma resposta visual ao avanço das atividades.

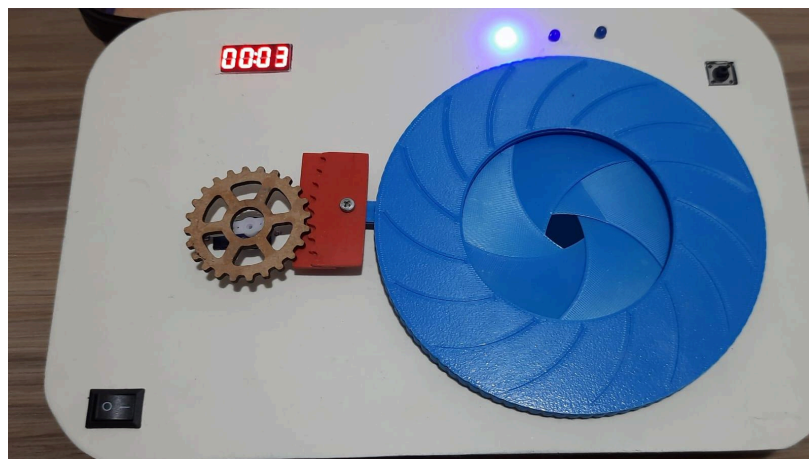
O mecanismo possui aproximadamente 150 mm de diâmetro e foi produzido por impressão 3D utilizando filamento PLA. Sua estrutura é composta por uma base fixa,

anel de acionamento, lâminas móveis e haste de transmissão, permitindo abertura e fechamento sincronizados por meio da rotação do anel principal.

O acionamento é realizado por um servomotor SG90 controlado pelo ESP32. O movimento é transmitido por duas engrenagens, convertendo a rotação do servo em deslocamento do anel da íris. Durante a montagem foram realizados ajustes para reduzir atritos, melhorar o alinhamento das engrenagens e garantir um funcionamento suave do mecanismo.

O posicionamento do servomotor foi programado para diferentes ângulos, permitindo que a abertura da íris ocorra de forma gradual conforme o progresso do usuário nas etapas do jogo.

Figura 3 - Montagem inicial do painel da Íris



Fonte: autoria própria (18/05/2026)

Durante a montagem do protótipo, foi observado que a instalação de todos os componentes eletrônicos na parte traseira do painel provocava um desequilíbrio na estrutura. O peso concentrado nessa região fazia com que a maleta tombasse quando aberta, comprometendo a estabilidade e a utilização do equipamento.

Para solucionar esse problema, foi realizada uma redistribuição dos componentes eletrônicos, transferindo a maior parte deles para a base da maleta. Essa alteração reduziu o centro de gravidade do conjunto, proporcionando melhor distribuição de massa e maior estabilidade durante a operação, sem comprometer o funcionamento do sistema ou a organização interna.

A Figura 4 e 5 apresenta a configuração final adotada, evidenciando a nova disposição dos componentes após as modificações realizadas.

Figura 4 - Parte posterior do painel e distribuição dos componentes na parte inferior da maleta.



Fonte: autoria própria (18/05/2026)

Figura 5 - Tampa que foi inserida para proteger os os componentes e resultado final.



Fonte: autoria própria (18/05/2026)

2.1.3 Sistema Eletrônico

O sistema eletrônico foi desenvolvido com o objetivo de controlar a íris mecânica e integrar os demais dispositivos do projeto. O microcontrolador ESP32 foi utilizado como unidade central de processamento devido à sua capacidade de processamento, conectividade Wi-Fi integrada e facilidade de programação. A partir dele são

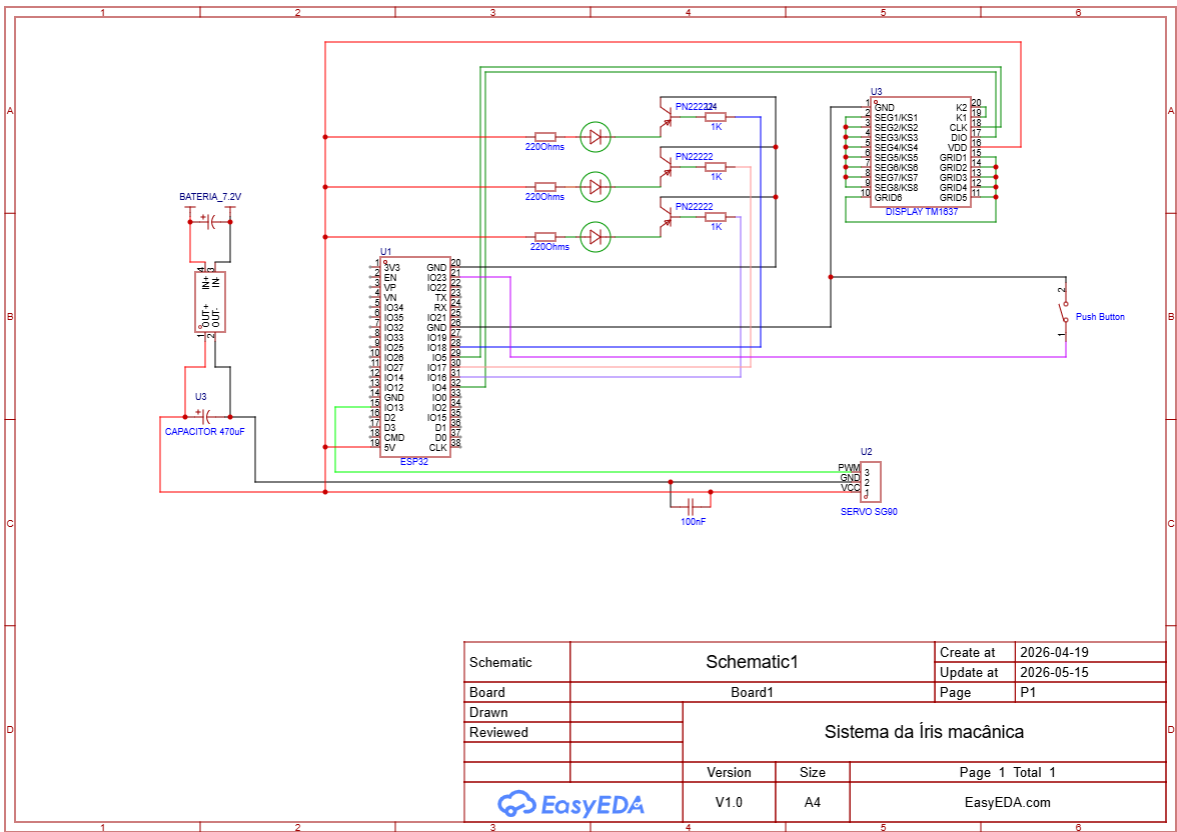
controlados o servomotor, os LEDs indicadores, o display digital e a comunicação entre os jogos e o painel.

A alimentação é realizada por duas baterias recarregáveis modelo 18650, cuja tensão é regulada para 5 V por meio de um conversor LM2596. Esse circuito fornece energia estável para todos os componentes eletrônicos, enquanto um capacitor eletrolítico de 1000 μF reduz oscilações de tensão causadas pelo acionamento do servomotor.

O sistema utiliza três LEDs para indicar visualmente as etapas concluídas pelo usuário e um display TM1637 para exibir o cronômetro durante a execução das atividades. Além disso, foi incorporado um botão de reinicialização, responsável por retornar o sistema ao estado inicial sem a necessidade de desligar a alimentação.

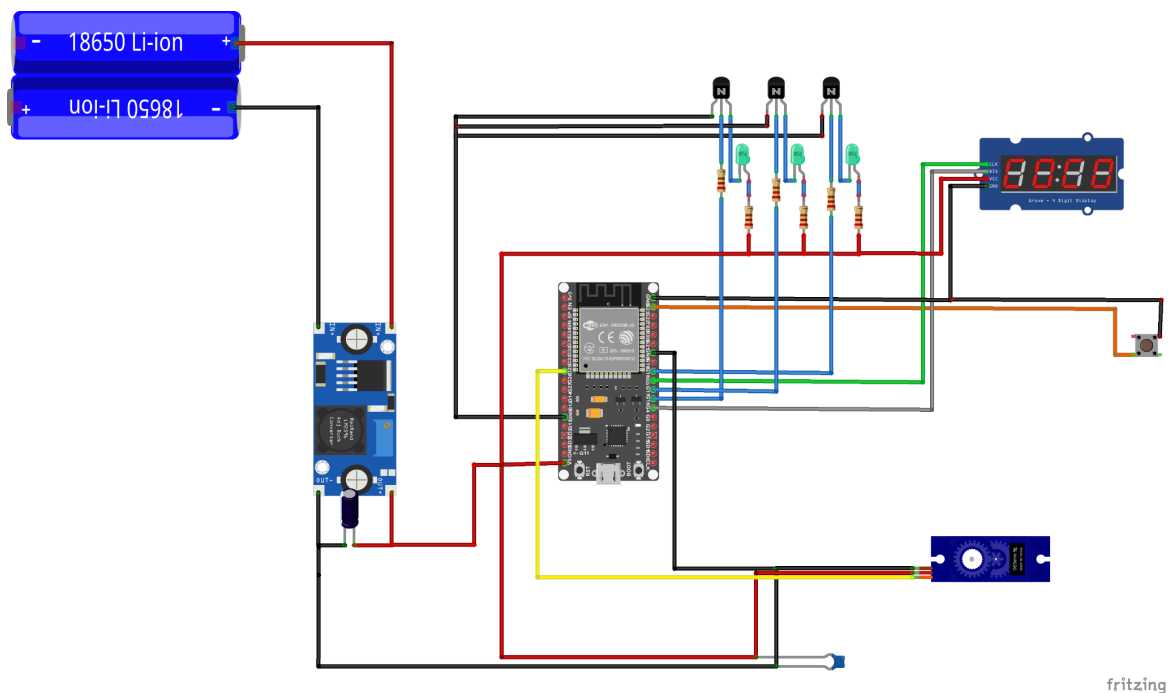
Para facilitar a montagem, o circuito foi inicialmente desenvolvido no EasyEDA e posteriormente montado em placa universal, utilizando o software Fritzing como referência para a disposição física dos componentes e conexões elétricas.

Figura 6 - Circuito Elétrico para o funcionamento do projeto.



Fonte: autoria própria feito no EasyEAD.

Figura 7 - Circuito dos componentes.



Fonte: autoria própria, feito no Fritzing.

2.1.4 Programação e Comunicação

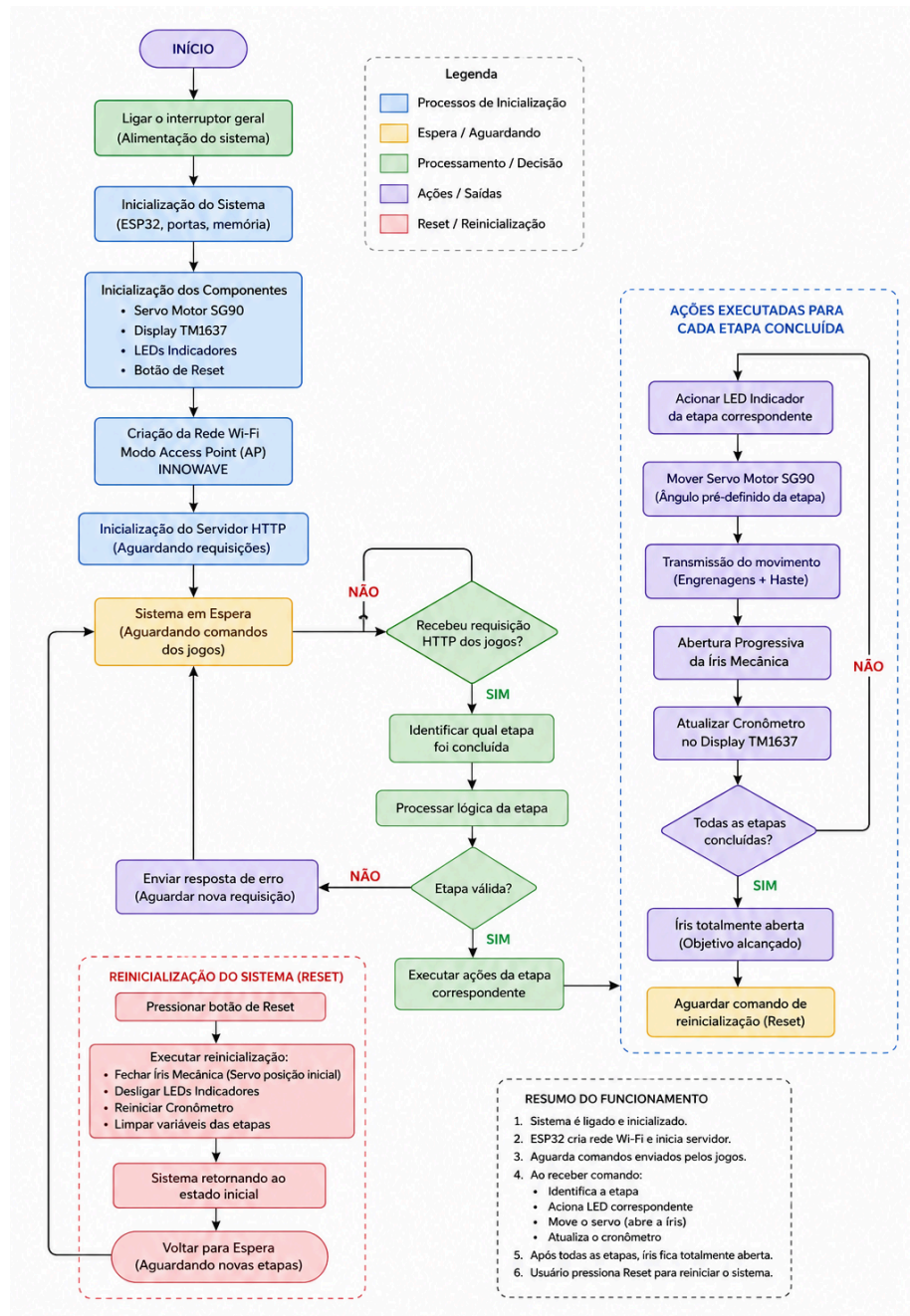
O controle do sistema foi desenvolvido na plataforma Arduino IDE utilizando linguagem C++. O código foi organizado em módulos independentes, responsáveis pela inicialização do sistema, comunicação Wi-Fi, controle do servomotor, acionamento dos LEDs e atualização do display digital. Essa estrutura facilita a manutenção do software e permite a implementação de novas funcionalidades.

Após a energização do equipamento, o ESP32 inicializa todos os dispositivos conectados e cria uma rede Wi-Fi local denominada INNOWAVE, operando em modo Access Point. Dessa forma, a comunicação entre os jogos e o painel ocorre sem necessidade de internet ou roteadores externos, tornando o sistema mais simples e portátil.

O ESP32 hospeda um servidor HTTP interno que recebe as requisições enviadas pelos jogos. Cada etapa concluída corresponde a um comando específico, que é processado pelo microcontrolador para atualizar o estado do sistema. Como resposta, o servomotor movimenta a íris para uma nova posição, os LEDs indicam o progresso alcançado e o display permanece exibindo o tempo de execução.

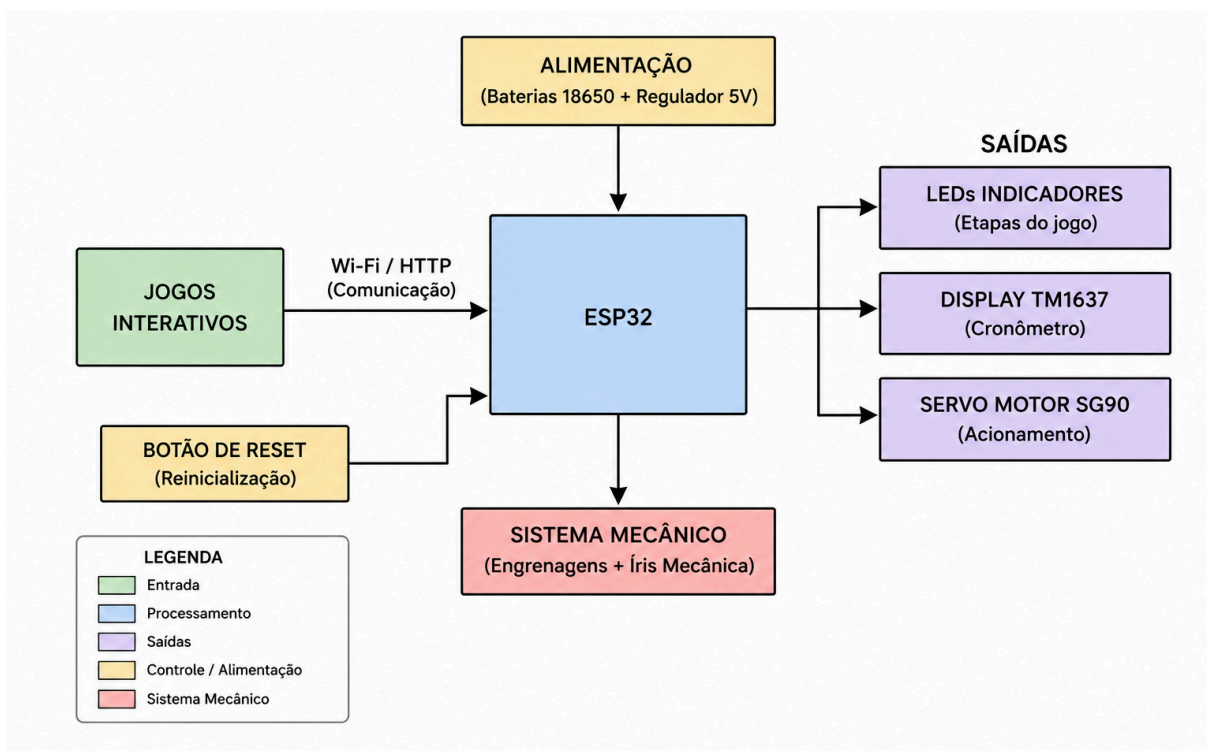
Ao término da atividade, o botão de reset permite reinicializar o equipamento. Quando acionado, o software retorna o servomotor à posição inicial, fecha completamente a íris, apaga os LEDs, reinicia o cronômetro e prepara o sistema para uma nova utilização. Essa rotina garante rapidez na reinicialização e maior praticidade durante os testes e demonstrações. No apêndice A encontra-se a programação completa.

Figura 8 - Fluxograma do funcionamento do sistema



Fonte: autoria própria, feito no Canva.

Figura 9 - Diagrama



Fonte: autoria própria, feito no Canva.

2.1.5 Componentes e tabela de valores

A tabela a seguir apresenta os principais componentes utilizados no desenvolvimento do projeto e valores médios estimados. Alguns dos componentes foram usados em projetos anteriores.

Tabela 1 - Custos totais

Componente	Quantidade	Valor Unitário (R\$)	Valor Total (R\$)
ESP32	1	35,00	35,00
Servo Motor SG90	1	18,00	18,00
Display TM1637	1	12,00	12,00
LEDs azuis	3	1,00	3,00
Transistor 2N2222A	3	1,50	4,50

Resistores diversos	6	0,20	1,20
Módulo Step Down LM2596	1	12,00	12,00
Baterias 18650	2	20,00	40,00
Capacitor 1000 μ F	1	3,00	3,00
Botão tátil (reset)	1	2,00	2,00
Interruptor geral	1	2,00	2,00
MDF 3 mm	4 placas	7,00	14,00
Ímãs de fixação	4	2,00	8,00
Cola de contato	1	18,00	18,00
Cola quente	5	5,00	5,00
Tecido para revestimento	—	0,00	0,00
Perfil de borracha	—	0,00	0,00
Zíper	1	10,00	10,00
Impressão 3D	—	40,00	40,00
Maleta de MDF	1	30,00	30,00
Tinta Spray e prime	2	15,00	30,00
Total			287,70
Total (esse semestre)			190,00

Fonte: Autoria própria (2026).

2.1.7 Sistema de Alimentação

O sistema de alimentação foi projetado para garantir autonomia e estabilidade durante a operação do equipamento. A energia é fornecida por duas baterias de íons de lítio

modelo 18650, enquanto um regulador LM2596 converte a tensão para 5 V, compatível com os dispositivos eletrônicos utilizados.

Com base nas especificações dos componentes, estimou-se um consumo médio de aproximadamente 540 mA, considerando o funcionamento simultâneo do ESP32, servomotor, display e LEDs.

A autonomia foi calculada pela expressão:

$$t = C / I$$

onde:

- t = autonomia (h);
- C = capacidade útil da bateria (mAh);
- I = corrente média do sistema (mA).

Considerando uma capacidade útil de aproximadamente 4180 mAh, obteve-se uma autonomia teórica de 7,7 horas. Em condições reais de utilização, esse valor pode variar entre 5 e 7 horas, dependendo da frequência de acionamento do servomotor e das perdas do regulador de tensão.

A potência média consumida pelo equipamento foi estimada utilizando a relação entre tensão e corrente:

$$P = V \cdot I$$

Adotando tensão de 5 V e corrente média de 0,54 A, a potência aproximada do sistema é de 2,7 W, indicando baixo consumo energético para um equipamento que integra sistemas mecânicos, eletrônicos e de comunicação.

2.1.8 Testes Realizados

Após a conclusão da montagem, foram realizados testes para verificar o funcionamento integrado dos sistemas mecânico, eletrônico e de comunicação. Os ensaios tiveram como objetivo validar o desempenho da íris mecânica, do circuito eletrônico, da comunicação Wi-Fi e da estrutura física do equipamento, garantindo que todos os componentes operassem de forma sincronizada.

Os testes foram divididos em quatro etapas: avaliação do mecanismo de abertura, verificação do circuito eletrônico, validação da comunicação entre os jogos e o painel e análise da estrutura da maleta. Durante os ensaios também foi observado o comportamento do sistema em funcionamento contínuo, verificando estabilidade e confiabilidade.

2.1.9 Desempenho Mecânico

O mecanismo da íris foi submetido a diversos ciclos de abertura e fechamento para verificar o funcionamento do sistema de transmissão. Durante os testes, as engrenagens apresentaram movimento uniforme, sem ocorrência de travamentos ou desalinhamentos significativos.

Pequenos ajustes no posicionamento do servomotor e das engrenagens foram realizados para reduzir atritos e melhorar a suavidade do movimento. Após essas correções, o conjunto apresentou funcionamento estável e repetitivo, mantendo o sincronismo entre o servomotor e a abertura da íris.

2.1.10 Desempenho Eletrônico

O circuito eletrônico foi avaliado durante o funcionamento completo do equipamento. O ESP32 respondeu corretamente aos comandos enviados pelos jogos, realizando o acionamento do servomotor, dos LEDs e do display conforme programado.

Também foi verificada a estabilidade da alimentação elétrica durante os acionamentos do servomotor. O regulador LM2596, aliado ao capacitor de filtragem, manteve a tensão estável, evitando reinicializações do microcontrolador e garantindo o funcionamento contínuo do sistema.

2.1.11 Comunicação do Sistema

A comunicação entre os jogos e o painel foi validada utilizando a rede Wi-Fi criada pelo ESP32 em modo Access Point. Cada etapa concluída enviou uma requisição HTTP ao microcontrolador, que processou o comando e executou as ações correspondentes.

Durante os testes, a comunicação ocorreu de forma estável e sem necessidade de conexão com a internet. O tempo de resposta foi suficiente para que a abertura da íris acompanhasse o avanço das etapas, proporcionando uma interação contínua entre o usuário e o equipamento.

2.1.12 Avaliação Estrutural

A estrutura da maleta também foi avaliada quanto à resistência e à organização interna dos componentes. O painel permaneceu firmemente fixado durante os testes, permitindo fácil acesso aos módulos eletrônicos quando necessário.

O sistema de fixação adotado mostrou-se adequado para a aplicação, mantendo a estabilidade do conjunto e facilitando operações de manutenção. Além disso, a disposição interna dos componentes favoreceu a organização do cabeamento e contribuiu para o correto funcionamento do equipamento.

2.1.13 Resultados Obtidos

Os testes demonstraram que o sistema atendeu aos requisitos definidos durante o desenvolvimento do projeto. A integração entre os sistemas mecânico, eletrônico e de comunicação ocorreu conforme o esperado, permitindo a abertura progressiva da íris em resposta às etapas concluídas nos jogos.

Entre os principais resultados obtidos destacam-se:

- funcionamento contínuo da íris mecânica;
- sincronização entre os jogos e o painel de controle;
- comunicação Wi-Fi estável;
- acionamento correto dos LEDs e do display;
- alimentação elétrica confiável;
- estrutura compacta e de fácil manutenção.

Tabela 2 - Apresenta um resumo dos principais testes realizados.

Teste	Resultado
Funcionamento da íris	Aprovado
Sistema de transmissão	Aprovado
Controle do servomotor	Aprovado
Comunicação Wi-Fi	Aprovado
Display e LEDs	Aprovado

Alimentação elétrica	Aprovado
Estrutura da maleta	Aprovado

De forma geral, o protótipo apresentou desempenho satisfatório, confirmando a viabilidade da solução desenvolvida e demonstrando que a integração dos diferentes módulos ocorreu de maneira eficiente.

2.1.14 CONCLUSÃO

O projeto permitiu desenvolver um sistema portátil capaz de integrar mecanismos mecânicos, eletrônicos e de comunicação em um único equipamento. A utilização do microcontrolador ESP32 como unidade central simplificou o controle do sistema, permitindo o gerenciamento da comunicação Wi-Fi, do servomotor, dos LEDs e do display digital de forma integrada. A adaptação da maleta, aliada à construção do painel e da íris mecânica, resultou em um equipamento compacto, funcional e de fácil transporte. Os testes realizados confirmaram o correto funcionamento da estrutura mecânica, da alimentação elétrica e da comunicação entre os jogos e o painel, demonstrando que os objetivos propostos foram alcançados. Além da construção do protótipo, o projeto proporcionou a aplicação prática de conhecimentos em programação embarcada, eletrônica, manufatura aditiva e integração de sistemas. A experiência adquirida durante o desenvolvimento contribuiu para a compreensão das etapas de projeto, montagem, testes e validação de um sistema mecatrônico.

Figura 10 - Projeto da maleta e painel finalizado.



Fonte: autoria própria.

2.2 Jogo 1 – Eletrônico

2.2.1 Concepção Inicial

O jogo eletrônico foi desenvolvido como um dos três desafios do projeto M.E.D. Lock, inspirado no jogo Genius. Sua finalidade é estimular memória, concentração e raciocínio lógico, permitindo que o usuário avance na abertura da maleta após reproduzir corretamente uma sequência luminosa. Na primeira versão, o protótipo validou a proposta de gamificação, porém apresentou limitações de ergonomia, organização elétrica e integração com o sistema central.

2.2.2 Diagnóstico da Primeira Versão

Durante as avaliações realizadas pela equipe e pelos docentes, verificou-se que o protótipo necessitava de melhorias na disposição dos componentes, organização do cabeamento, facilidade de manutenção e comunicação com a maleta. Embora o algoritmo do jogo funcionasse corretamente, a integração ainda não era suficientemente robusta para garantir estabilidade durante demonstrações contínuas.

Figura 11 - Versão Final



Fonte: Autoria própria (2026).

2.2.3 Evolução Estrutural

Com base nas recomendações recebidas, a estrutura foi reprojetaada. A caixa passou a possuir formato quadrado, melhor distribuição dos botões luminosos e centralização dos comandos de Start/Reset e da chave gangorra, tornando a utilização mais intuitiva. A organização interna também foi revisada para facilitar futuras manutenções. A figura 12 apresenta a versão finalizada com as melhorias.

Figura 12 - Versão Final

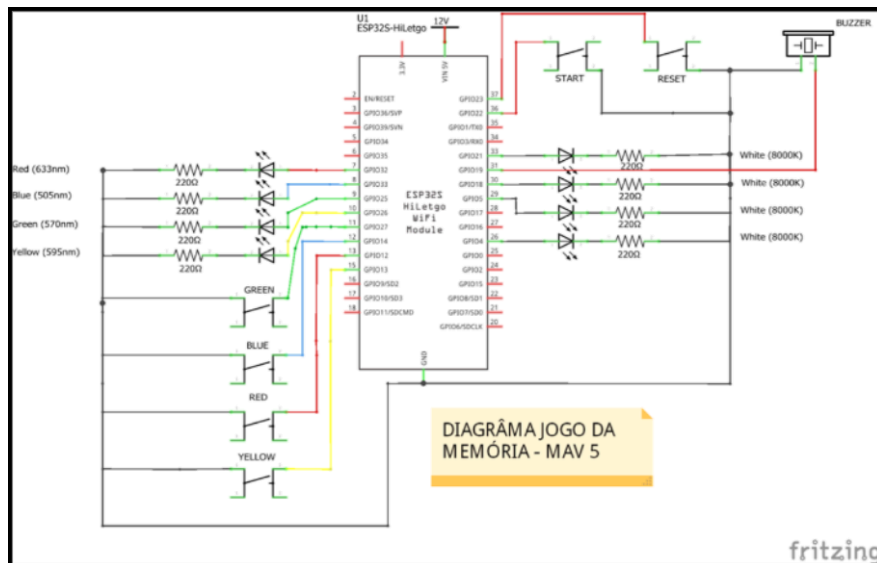


Fonte: Autoria própria (2026).

2.2.4 Evolução Eletrônica

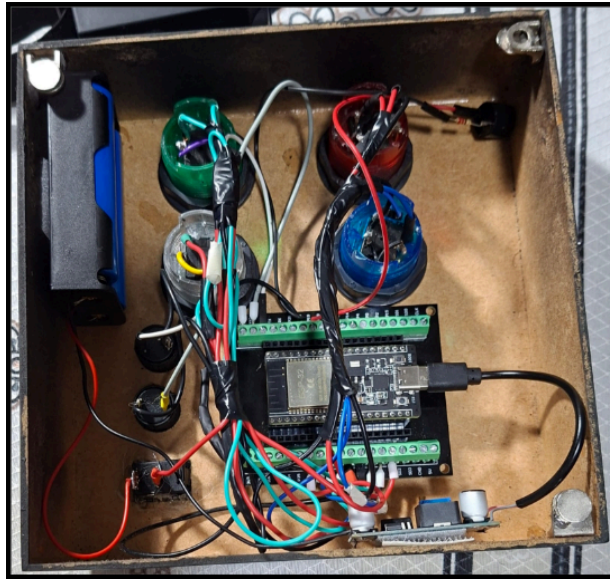
A parte eletrônica recebeu uma reorganização completa da fiação e dos módulos, reduzindo interferências e aumentando a confiabilidade do circuito. Foi elaborado um novo diagrama elétrico, utilizado como referência para montagem e futuras manutenções. As figuras 13 e 14 apresentam o diagrama e a montagem atual

Figura 13 - Diagrama Elétrico



Fonte: Autoria própria (2026).

Figura 14 - Eletrônica Atual



Fonte: Autoria própria (2026).

2.2.5 Integração com a Maleta

A principal evolução desta etapa foi a implementação da comunicação entre o jogo e o sistema central do M.E.D. Lock. Foi adicionada comunicação via localhost/HTTP utilizando ESP32, permitindo que, após a conclusão do desafio, o jogo enviasse automaticamente o comando responsável por liberar uma etapa da abertura da íris mecânica. Essa modificação eliminou falhas observadas na primeira versão e tornou o funcionamento totalmente integrado. As figuras 15 e 16 apresentam o código da solução.

Figura 15 - Bibliotecas

```
#include <Arduíno.h>
#include <WiFi.h>
#include <HTTPClient.h>

// Configurações da rede do Servidor INNOWAVE
const char* ssid = "INNOWAVE";
const char* password = "SSID1234";

// IP padrão do ESP32 em modo Access Point
const char* serverName = "http://192.168.4.1";

#define NUM_CORES 4
#define NUM_FASES 4
#define TAM_SEQUENCIA 5

// Botões luminosos (restaurado com 4 cores)
int botoes[NUM_CORES] = { 13, 12, 14, 27 };
int ledsCores[NUM_CORES] = { 26, 25, 33, 32 };

// LEDs indicadores da sequência/fase (4 fases)
int ledsFase[NUM_FASES] = { 4, 5, 18, 21 };

int botaoStart = 22;
int botaoReset = 23;
int buzzer = 19;

// Trava mantida
int trava = 2;

// Tons restaurados
int tons[NUM_CORES] = { 261, 329, 392, 523 };

int sequencia[NUM_FASES][TAM_SEQUENCIA];
int faseAtual = 0;
```

Fonte: Autoria própria (2026).

Figura 16 - Bibliotecas

```
//----Cliente Padrao
void enviarComando(bool acender) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    String requestPath = "/ledMem/on";
    String fullUrl = String(serverName) + requestPath;

    Serial.print("Enviando: ");
    Serial.println(fullUrl);

    http.begin(fullUrl);
    int httpResponseCode = http.GET();

    if (httpResponseCode > 0) {
      Serial.print("Resposta do Servidor: ");
      Serial.println(httpResponseCode);
    } else {
      Serial.print("Erro na comunicação: ");
      Serial.println(httpResponseCode);
    }
    http.end();
  } else {
    Serial.println("Erro: Wi-Fi desconectado");
  }
}

void aguarde(unsigned long tempo) {
  unsigned long tempoAnterior = millis();
  while (millis() - tempoAnterior < tempo) {
    // O processador fica preso aqui, mas você poderia colocar
    // outras verificações rápidas aqui dentro se necessário.
  }
}
```

Fonte: Autoria própria (2026).

2.2.6 Testes de autonomia

A tabela a seguir apresenta os resultados dos testes de autonomia e recarga do jogo da memória, simulando o período na apresentação da feira de soluções.

Tabela 3 - Período de testes de autonomia e recarga

	PERÍODO DE USO PARA TESTES	PERÍODO DE RECARGA
	Das 17:30 às 23:51	Das 23:55 às 02:00
TOTAL TEMPO	06:21	01:05

Fonte: Autoria própria (2026).

2.2.7 Custos

A tabela a seguir contém os componentes e custos do protótipo do jogo da memória. Essa tabela demonstra a viabilidade econômica da solução e evidencia que a maior parte dos componentes utilizados possui baixo custo e ampla disponibilidade comercial.

Tabela 4 - Materiais e custos

LISTA DE MATERIAIS - JOGO DA MEMÓRIA			
ITEM	QTD	DESCRIÇÃO	CUSTO
1	1	CAIXA DE MDF 170X170X50	R\$ 6,00
2	1	ESP 32	R\$ 0,00
3	1	PLACA DE EXPANSÃO ESP 32	R\$ 0,00
4	4	BOTÕES LUMINOSOS MICROSWITCH*	R\$ 11,69
5	2	BOTÃO CHAVE TACTIL	R\$ 0,00
6	1	CHAVE GANGORRA ON/OFF	R\$ 0,00
7	2	BATERIAS 18650	R\$ 38,75
8	1	CASE PARA BATERIA 18650	R\$ 15,90
9	1	MÓDULO STEPUP MT3608	R\$ 19,90
10	1	FIOS PARA PROTOTIPAGEM	R\$ 0,00
11	1	ORGANIZADOR DE FIOS	R\$ 11,22
TOTAL	16	-	R\$ 103,46
OBSERVAÇÕES		Utilizamos quatro unidades do botão luminoso, porém houve a necessidade de compra de um novo botão, explicando o valor.	

Fonte: Autoria própria (2026).

2.2.8 Resultados

Após as melhorias implementadas, o jogo apresentou maior estabilidade operacional, melhor organização física, facilidade de manutenção e comunicação confiável com a maleta. Os testes realizados demonstraram que o sistema responde corretamente às entradas do usuário, transmite o comando de conclusão sem falhas e atende aos objetivos definidos para o módulo eletrônico.

2.2.9 Conclusão

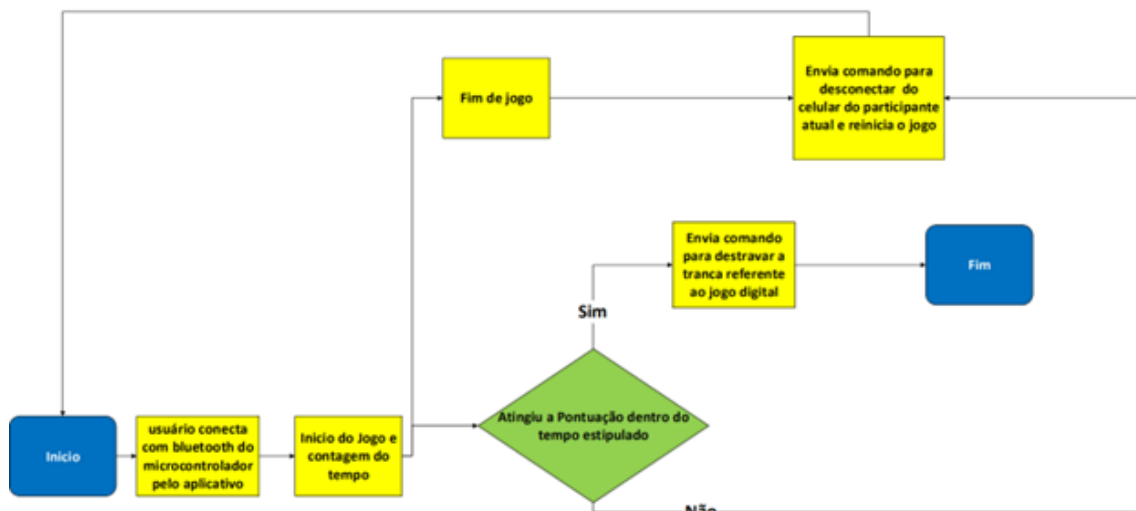
A evolução do jogo da memória representa um processo contínuo de aperfeiçoamento de um protótipo funcional para uma solução integrada. As modificações estruturais, eletrônicas e de software aumentaram a confiabilidade do sistema, melhoraram a experiência do usuário e permitiram sua integração definitiva ao projeto M.E.D. Lock. Além de atender aos objetivos técnicos, o desenvolvimento proporcionou aos integrantes experiência prática em eletrônica embarcada, integração de sistemas, organização de projetos e desenvolvimento de produtos.

2.3 Jogo 2 – Digital

O Jogo 2, será digital fazendo-se necessário o uso de um tablet. Inicialmente o aplicativo havia sido desenvolvido para celular, porém, por motivos que serão abordados mais adiante nesta seção houve mudança para tablet, seria usado a plataforma de desenvolvimento “Scratch”, porém por questão de facilidade foi escolhido a plataforma MIT APP Inventor.

O fluxograma funcional abaixo demonstra o funcionamento da aplicação desenvolvida. Ao atingir a pontuação adequada será aberta parcialmente a íris que sela a caixa.

Figura 17 - Fluxograma funcional – Jogo 2 - Digital



Fonte: autoria própria (08/04/2026)

2.3.1 - Contextualização

Na etapa inicial do projeto, foi desenvolvido aplicativo com um jogo de tetris para celular, no qual o usuário precisa atingir 10 pontos dentro de um tempo limitado de 3 minutos (180 segundos). A comunicação entre o aplicativo e a caixa eletrônica foi implementada por meio da tecnologia Bluetooth.

Ao término da execução do jogo, uma vez atingido o critério de pontuação definido, o aplicativo envia um comando específico ao sistema de controle da caixa eletrônica via

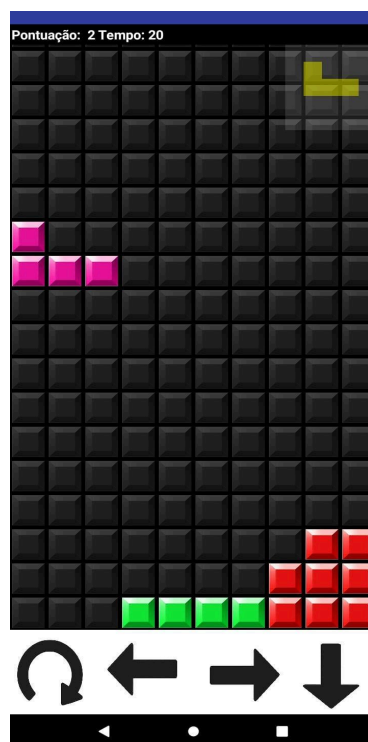
bluetooth. Esse comando é interpretado pelo dispositivo, que realiza o acionamento de um mecanismo de liberação, permitindo a abertura de uma das travas existentes no sistema.

2.3.1.1 - Alterações - Interface de Usuário

Posteriormente, em decorrência de solicitações dos docentes orientadores do projeto, foi necessária a adaptação da aplicação para funcionamento em dispositivos do tipo tablet. Essa mudança implicou na reestruturação parcial da interface e na adequação do sistema para diferentes dimensões de tela e modos de interação, mantendo, entretanto, a integridade das funcionalidades previamente implementadas, apenas ajustando a comunicação do jogo com a caixa eletrônica.

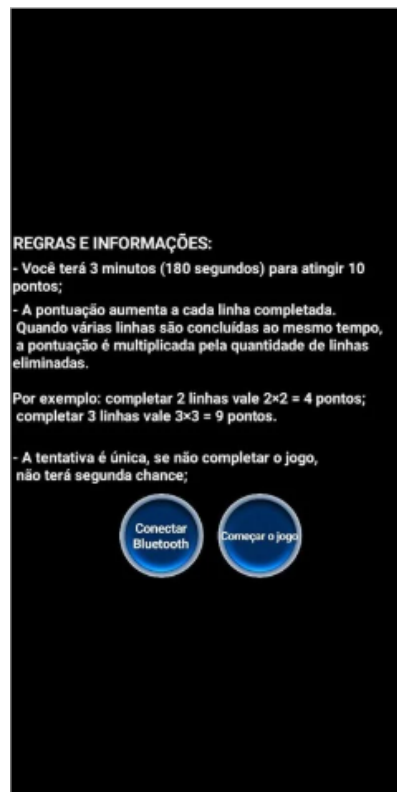
As figuras 24 e 25 demonstram como a tela era na utilização pelo celular:

Figura 18 - Tela do Celular – Tela de Jogo - Jogo Digital



Fonte: autoria própria (08/04/2026)

Figura 19 - Tela do Celular – Tela Inicial - Jogo Digital



Fonte: autoria própria (08/04/2026)

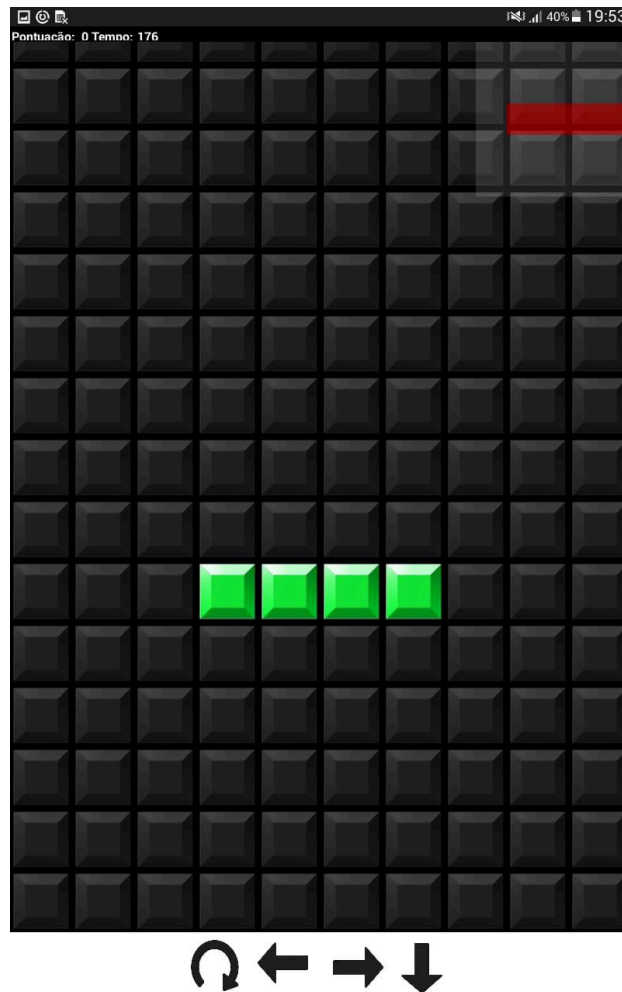
As figuras 26 e 27 abaixo demonstram como está a tela atual utilizando o tablet:

Figura 20 - Tela do Tablet – Tela Inicial - Jogo Digital



Fonte: autoria própria (08/04/2026)

Figura 21 - Tela do Tablet V1– Tela de Jogo - Jogo Digital



Fonte: autoria própria (08/04/2026)

2.3.1.2 - Alterações - Comunicação e testes

Durante a fase de desenvolvimento da comunicação do projeto, foram realizados testes experimentais com o objetivo de validar a comunicação entre o aplicativo desenvolvido e o hardware embarcado baseado no microcontrolador ESP32.

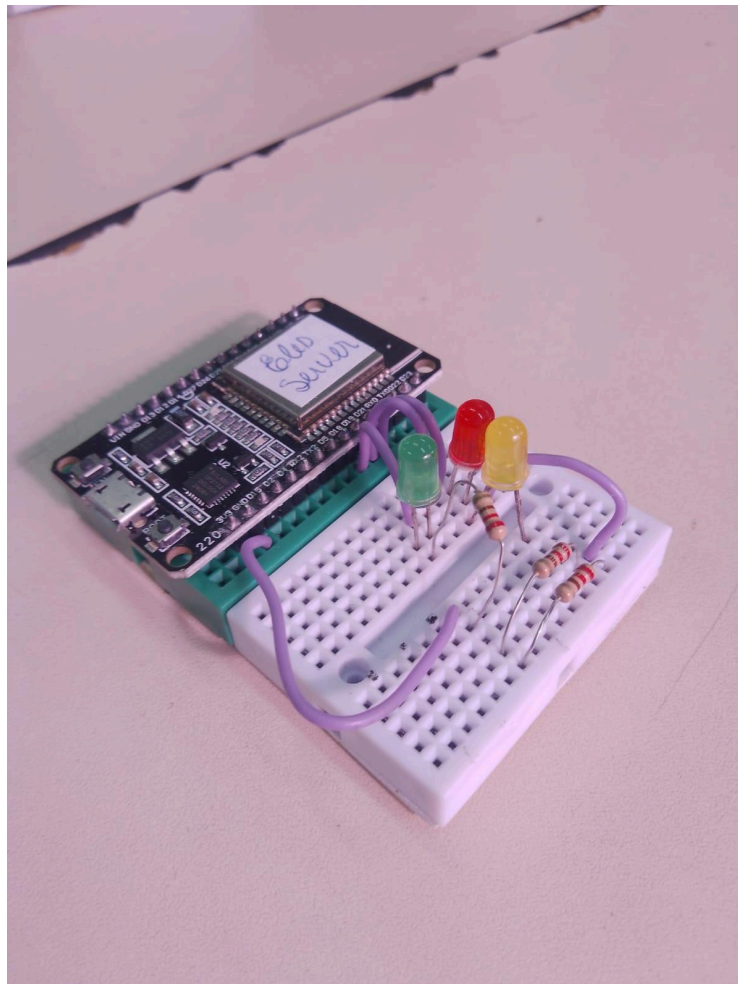
Inicialmente, foi implementado um sistema de comunicação em rede local (LAN), no qual o ESP32 opera como servidor Wi-Fi, permitindo que dispositivos clientes, como tablets ou smartphones, se conectem diretamente à sua rede. O aplicativo do jogo foi configurado para interagir com esse servidor, enviando comandos por meio de requisições via URL, caracterizando uma comunicação baseada em protocolo HTTP.

Como etapa inicial de validação, foi desenvolvido um circuito eletrônico simples contendo um LED, utilizado como elemento de resposta para os testes de comunicação. O objetivo dessa etapa foi verificar a integridade do envio e recebimento de comandos entre o software e o hardware.

Os testes demonstraram sucesso, uma vez que os comandos enviados pelo aplicativo foram corretamente processados pelo ESP32, resultando no acionamento do LED conforme esperado.

Abaixo a imagem mostra o hardware utilizado no teste:

Figura 22 - Hardware utilizado no teste de comunicação - Jogo Digital

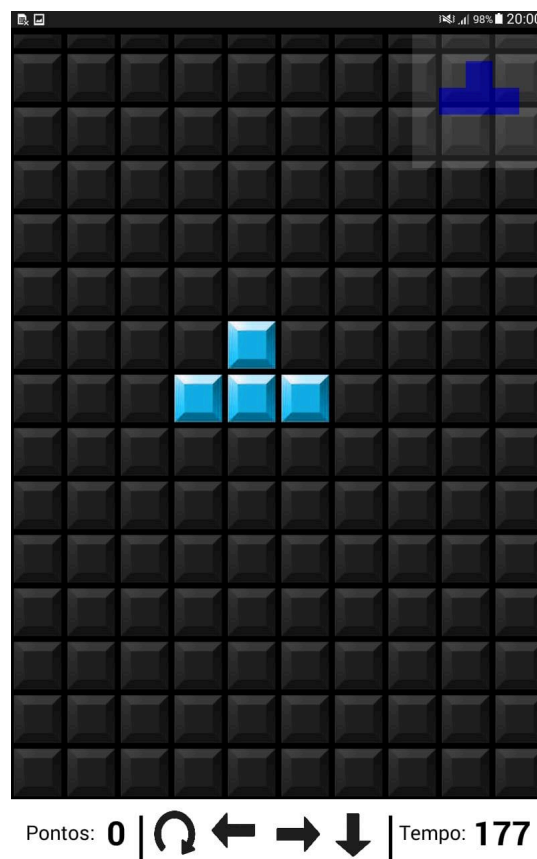


Fonte: autoria própria (06/05/2026)

Paralelamente, foram realizadas melhorias na interface do usuário (UI) do aplicativo, com foco na otimização da experiência do usuário. As modificações tiveram como objetivo reduzir a poluição visual e organizar de forma mais eficiente as informações exibidas, como tempo de jogo, pontuação e área de interação. Essas alterações contribuíram para uma interface mais intuitiva e funcional.

Adiante a imagem mostra a interface atual:

Figura 23 - Tela do Tablet V2 – Tela de Jogo - Jogo Digital



Fonte: autoria própria (06/05/2026)

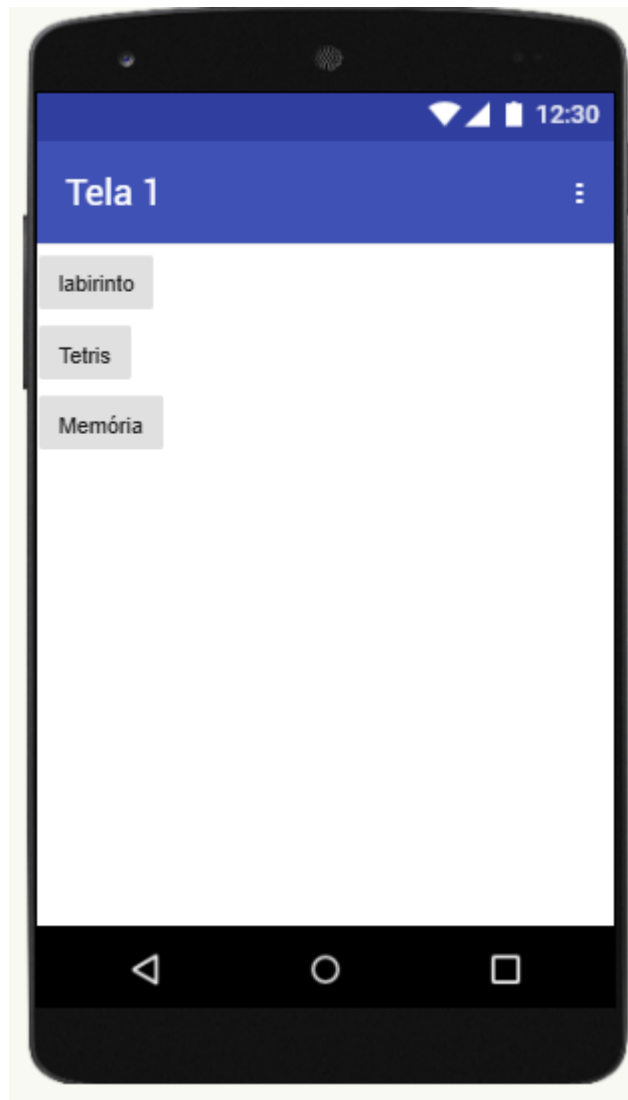
Como evolução do sistema, foi incorporada ao aplicativo uma extensão denominada TaifunWifi, desenvolvida por um membro da comunidade do MIT App Inventor. Essa extensão permitiu a implementação de um mecanismo de conexão automática à rede Wi-Fi do ESP32. Com isso, sempre que o servidor é iniciado, o aplicativo já está pré

programado para realizar a conexão de forma automática, eliminando a necessidade de configuração manual por parte do usuário e aumentando a eficiência operacional do sistema, apenas deve-se se atentar a versão android, pois na própria página de download desta extensão, cujo link está presente na seção de referências deste documento, destaca cautela no método para conexão wifi automática por aprovação do usuário para quando estiver usando em android 10 em diante, esta extensão está gratuita até sua 15ª versão, versões mais recentes da extensão são pagas.

Adicionalmente, foi desenvolvido um aplicativo auxiliar específico para testes de envio de comandos. Esse aplicativo possuía uma interface simplificada baseada em botões, permitindo o disparo direto de comandos para o hardware. Durante os testes, verificou-se que os comandos foram corretamente transmitidos e executados pelo sistema físico.

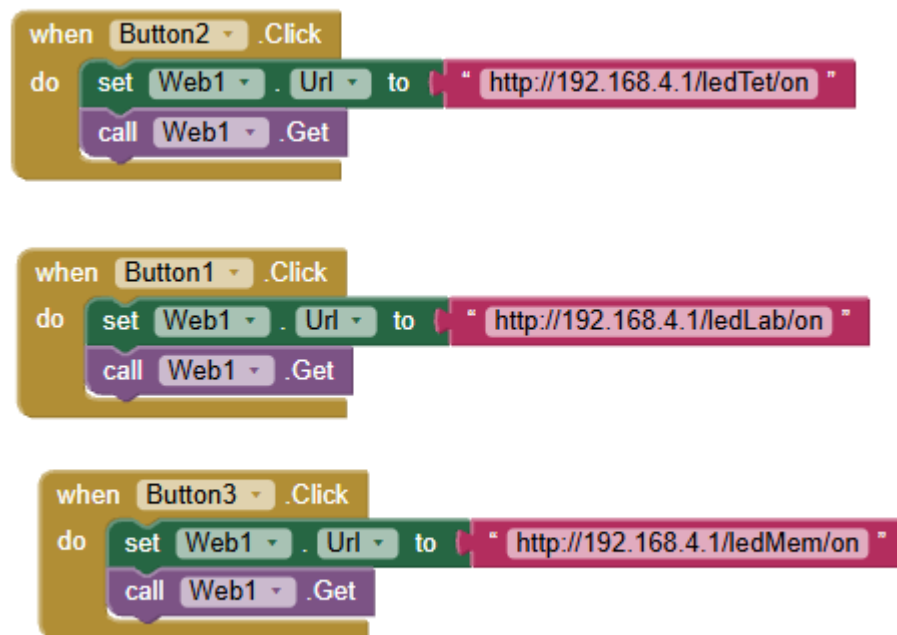
Abaixo nas figuras 30 e 31 estão em destaque detalhes da aplicação de teste de comunicação:

Figura 24 - Tela da aplicação - Jogo Digital



Fonte: autoria própria (06/05/2026)

Figura 25 - Programação em blocos da aplicação - Jogo Digital



Fonte: autoria própria (06/05/2026)

Como aplicação prática dessa comunicação, foi testado o acionamento de um mecanismo integrado a uma maleta, responsável pela abertura de uma estrutura do tipo íris, onde será armazenado o prêmio do jogo. Os testes demonstraram que, ao acionar os comandos pelo aplicativo, o sistema embarcado recebeu e processou corretamente as instruções, resultando na execução do movimento de abertura da íris de forma eficiente.

Abaixo a imagem destaca um dos testes finais de comunicação realizados:

Figura 26 - Testes finais de comunicação - Jogo Digital



Fonte: autoria própria (06/05/2026)

Dessa forma, os testes realizados validaram com sucesso a arquitetura de comunicação proposta, evidenciando a confiabilidade da integração entre software e hardware no contexto do projeto.

2.3.2 - Suporte

Foi desenvolvido, por meio de software de desenho assistido por computador (CAD), um suporte destinado ao posicionamento do tablet em uma superfície adequada durante a utilização do jogo. Essa solução foi concebida com o objetivo de proporcionar maior ergonomia e conforto ao usuário, eliminando a necessidade de segurar o dispositivo com as mãos durante a atividade, reduzindo o esforço físico e favorecendo uma experiência de uso mais estável e confortável.

A imagem a seguir exhibe o suporte que foi impresso e teve acabamento com tinta preta:

Figura 27 - Suporte para o tablet



Fonte: autoria própria (06/05/2026)

Qualquer pessoa poderá replicar o jogo digital utilizando os tutoriais e exemplos nas referências sobre o jogo digital.

2.4 - Jogo 3 – Eletromecânico: Integração do Projeto e Jogo Labirinto

2.4.1 - Integração do Projeto

Proposta de Integração do Jogo (MED Lock): O objetivo principal desta etapa do projeto foi estabelecer uma comunicação sem fio eficiente para o sistema MED Lock. A ideia central foi integrar de forma wireless os três jogos desenvolvidos: o Jogo da Memória, o Jogo Tetris e o Jogo Labirinto.

A lógica de funcionamento foi pensada para que cada jogo funcionasse como uma etapa independente. Ao concluir o seu respectivo desafio, o jogo envia um sinal de confirmação para a central. Esse sinal atua diretamente no circuito da Maleta, acionando o destravamento mecânico da Íris de forma proporcional — ou seja, a maleta vai se abrindo à medida que o jogador avança e conclui as etapas específicas da MED Lock.

2.4.2 - Pesquisa e Desenvolvimento

Investigação Inicial e Conflitos de Hardware: O desenvolvimento começou com uma pesquisa sobre os métodos de comunicação sem fio suportados pela ESP32, focando inicialmente em Bluetooth e Wi-Fi.

Como o projeto inclui um Tablet dedicado para rodar o jogo Tetris, tentou-se usar as bibliotecas de Bluetooth e Wi-Fi juntas no mesmo microcontrolador. Porém, surgiu um problema prático: houve muito conflito de comunicação. Durante os testes, notou-se que a biblioteca do Bluetooth tem prioridade sobre a de Wi-Fi na placa, o que gerava instabilidade e quedas de conexão. Por essa razão, foi necessário buscar outra solução.

2.4.3 - Redirecionamento da Solução e Primeiros Testes

Como a maior dificuldade de comunicação estava no Tablet com o jogo Tetris, a pesquisa foi direcionada para o uso exclusivo do Wi-Fi através de uma rede de internet.

Nos primeiros testes, foi criado na ESP32 da Maleta um sistema Servidor para receber as informações e realizar as funções de comando da Íris. Nessa configuração inicial, o jogo Tetris acessava a página do Servidor da Maleta e enviava o comando informando que o jogo havia sido concluído, liberando a Íris para abrir proporcionalmente.

2.4.4 - Ajuste de Arquitetura e Independência de Rede

A intenção do grupo é deixar o projeto pronto para ser utilizado em feiras e eventos da Fatec. Por isso, o Professor Orientador aconselhou o uso de uma rede que não dependesse de acesso à internet externa, facilitando o transporte e o uso em qualquer local.

Para resolver essa necessidade, a pesquisa avançou sobre a arquitetura de Servidor e Cliente. Primeiramente, os testes ainda dependiam de uma internet externa para funcionar. Continuando os estudos, encontrou-se uma solução mais adequada: a criação de uma rede local privada, usando o modo Access Point da própria ESP32 da Maleta. Dessa forma, o sistema passou a funcionar de forma totalmente independente de internet externa.

O módulo ESP32 pode ser configurado para operar como um servidor web local (SANTOS; SANTOS, [s.d.]).

2.4.5 - Servidor / Cliente: Códigos Base do Sistema

Os códigos em linguagem C++ utilizados para estruturar a comunicação local do projeto estão descritos na íntegra a seguir.

Figura 23 - Código Base do Servidor (ESP32 da Maleta)

Figura 28 - Código Base do Servidor (ESP32 da Maleta)

```

1  /*
2  *** MALETA (LocalHost)
3  */
4  #include <WiFi.h>
5  #include <WebServer.h>
6
7  const int ledPinLab = 18;
8  const int ledPinTet = 19;
9  const int ledPinMem = 21;
10 const char* ssid = "INNOWAVE";
11 const char* password = "SSID1234";
12
13 WebServer server(80);
14
15 void setup() {
16   WiFi.softAP(ssid, password);
17   IPAddress IP = WiFi.softAPIP();
18   server.on("/", handleRoot);
19   server.on("/ledLab/on", ledPinLabOn); //Jogo Labirinto
20   server.on("/ledTet/on", ledPinTetOn); //Jogo Tetris
21   server.on("/ledMem/on", ledPinMemOn); //Jogo Memória
22   pinMode(ledPinLab, OUTPUT);
23   pinMode(ledPinTet, OUTPUT);
24   pinMode(ledPinMem, OUTPUT);
25   digitalWrite(ledPinLab, LOW);
26   digitalWrite(ledPinTet, LOW);
27   digitalWrite(ledPinMem, LOW);
28 }
29
30 void loop() {
31   server.handleClient();
32 }
33
34 void handleRoot() {
35   String html = "<!DOCTYPE html><html>";
36   html += "<head><meta name='viewport' content='width=device-width, initial-scale=1'>";
37   html += "<link rel='icon' href='data:/'>";
38   html += "<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align: center;};";
39   html += ".button { background-color: #4CAF50; border: none; color: white; padding: 16px 40px;";
40   html += "text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;};";
41   html += ".button2 {background-color: #555555;}</style></head>";
42   html += "<body><h1>INNOWAVE MED LOCK</h1>";
43   html += "<body><h1>*Destrave a Caixa*</h1>";
44   html += "<p><a href='/ledTet/on'><button class='button'>CONCLUIR</button></a></p>";
45   html += "</body></html>";
46   server.send(200, "text/html", html);
47 }
48
49 void ledPinLabOn() { //para cada jogo montar uma etapa
50   digitalWrite(ledPinLab, HIGH);
51   server.sendHeader("Location", "/");
52   server.send(303);
53 }
54
55
56 void ledPinTetOn() {
57   digitalWrite(ledPinTet, HIGH);
58   // Redireciona o navegador de volta para a raiz "/"
59   server.sendHeader("Location", "/");
60   server.send(303);
61 }
62
63 void ledPinMemOn() { //para cada jogo montar uma etapa
64   digitalWrite(ledPinMem, HIGH);
65   server.sendHeader("Location", "/");
66   server.send(303);
67 }
68

```

Fonte: autoria própria (18/05/2026)

2.4.5.1 - Análise do Código do Servidor

Linhas de 4 a 13: Iniciam-se as bibliotecas a serem usadas (WiFi.h e WebServer.h). As configurações de rede indicam o nome da rede, a senha e o endereço IP. Cria-se um servidor web para gerenciar as requisições e respostas HTTP na porta 80.

Linhas de 15 a 28: Cria-se um access point para acessar a própria rede local da ESP32. Em seguida, o endereço IP é indicado e, especificamente nas linhas 19, 20 e 21, traçam-se as rotas de acesso para os jogos: **/ledMem/on** para o Jogo da Memória, **/ledTet/on** para o Jogo Tetris e **/ledLab/on** para o Jogo Labirinto. Também define os pinos como saída (OUTPUT) e inicia todos em nível baixo (LOW).

Linhas de 30 a 47: Cria-se o loop que gerencia os clientes e estrutura uma página HTML na ESP32 para facilitar o acesso e o gerenciamento visual através de botões.

Linhas de 49 a 68: Define a lista de comandos a serem executados para cada rota traçada (Jogo Labirinto, Jogo Tetris e Jogo da Memória), alterando o estado do pino para nível alto (HIGH). Há também um comando para redirecionar o navegador de volta à página inicial.

Figura 29 - Código Base Cliente (Usado nos Jogos Memória e Labirinto)

```
1  /*
2  *** Código Padrao para os Jogos na Esp32.
3  */
4  #include <WiFi.h>
5  #include <HTTPClient.h>
6
7  const char* ssid = "INNOWAVE";
8  const char* password = "SSID1234";
9  const char* serverName = "http://192.168.4.1";
10
11 void setup() {
12     WiFi.begin(ssid, password);
13     while (WiFi.status() != WL_CONNECTED) {
14         delay(500);
15     }
16 }
17
18 void loop() {
19     enviarComando(true); //-> Principal funcao ao terminar o jogo
20 }
21
22 void enviarComando(bool acender) { //-> deve entrar no final do codigo do Jogo Memoria
23     if (WiFi.status() == WL_CONNECTED) {
24         HTTPClient http;
25         String requestPath = "/ledMem/on"; // para jogo memoria
26         String fullUrl = String(serverName) + requestPath;
27         http.begin(fullUrl);
28         int httpResponseCode = http.GET();
29         http.end();
30     }
31 }
```

Fonte: autoria própria (18/05/2026)

2.4.5.2 - Análise do Código Cliente

Linhas 5 e 6: Iniciam-se as bibliotecas a serem usadas (WiFi.h e HTTPClient.h).

Linhas de 7 a 9: Configurações de rede, indicando o nome da rede, a senha e o endereço IP de destino do servidor (http://192.168.4.1).

Linhas de 11 a 16: Realiza a conexão com o servidor e faz uma verificação contínua para garantir que a conexão esteja estável.

Linhas de 18 a 20: Executam a função no loop para chamar a void enviarComando(true), que é a principal função responsável por enviar o sinal ao Servidor indicando que o jogo foi concluído com sucesso.

Linhas de 22 a 31: São as instruções da void enviarComando(true), que verifica se a conexão com o Servidor está ativa. Na linha 22, o caminho (requestPath) deve ser

alterado de acordo com o jogo que está sendo usado: `/ledMem/on` para o Jogo da Memória, `/ledTet/on` para o Jogo Tetris ou `/ledLab/on` para o Jogo Labirinto.

2.4.6 - Integrando cada Jogo à Maleta

A integração prática de cada desafio à estrutura da maleta seguiu o seguinte padrão.

Jogo Memória: Funciona em uma ESP32 configurada como Cliente. Ao terminar a lógica interna do jogo, o código chama a função de envio e faz uma requisição para a rota `/ledMem/on`, informando a central e acionando o pino correspondente.

Jogo Tetris: Este jogo roda em um Tablet. A integração foi feita conectando o Wi-Fi do tablet diretamente à rede "INNOWAVE" gerada pela Maleta. Quando o jogador conclui o objetivo no aplicativo, o sistema acessa a rota `/ledTet/on` do Servidor da Maleta para liberar a Íris proporcionalmente.

Jogo Labirinto: Utiliza uma ESP32 dedicada atuando como Cliente. Assim que os sensores do labirinto detectam que o percurso foi finalizado, a placa envia o sinal para o servidor através da rota `/ledLab/on`, completando mais uma etapa de abertura da maleta.

2.4.7 - Observações

Independência de Internet: A criação da rede local pela própria ESP32 resolveu o problema de transporte do projeto para feiras e eventos na Fatec, pois o sistema funciona perfeitamente sem depender de roteadores externos ou sinal de internet.

Solução do Conflito: Deixar de usar o Bluetooth eliminou as falhas e travamentos que aconteciam no processamento da ESP32, permitindo que a placa responda rapidamente às requisições Wi-Fi dos jogos.

2.4.8 - Conclusão

A organização do sistema em uma arquitetura de Servidor e Cliente trouxe a solução necessária para a integração da maleta MED Lock. Ao criar uma rede local própria na ESP32 da Maleta, o projeto ficou independente, portátil e estável. O objetivo foi alcançado: os jogos (Memória, Tetris e Labirinto) conseguem rodar suas rotinas e

enviar, via Wi-Fi, o sinal de conclusão, permitindo o destravamento automatizado e gradual da Íris da Maleta.

2.4.9 - Jogo Labirinto: Objetivo do Projeto

O objetivo do Jogo Labirinto é integrar conceitos de mecânica e eletrônica em um desafio de destreza. O jogador deve guiar uma bolinha de aço cromo desde o ponto de partida até o objetivo final do circuito. Ao longo do percurso, existem diversos obstáculos na forma de buracos; caso a bolinha caia em um deles, o percurso é interrompido, exigindo que o jogador reinicie o desafio.

2.4.10 - Funcionamento

O jogo opera de forma autônoma através da detecção eletrônica da bolinha de aço. Quando o jogador consegue desviar dos obstáculos e levar a bolinha até o final do percurso, um sensor posicionado no objetivo final detecta a sua presença.

Neste instante, a ESP32 local processa a informação e envia um sinal via Wi-Fi (usando a rede local descrita anteriormente) para a ESP32 Servidora da Maleta principal. Ao receber essa confirmação de término, a central aciona o motor da Íris, promovendo a abertura proporcional da fechadura da MED Lock.

2.4.11 - Circuito e Operação (Lista de Componentes)

Tabela 2.4A- Custo Total do Projeto

Componente	Quantidade	Valor Unitário (R\$)
ESP32 DevKit	1	Projeto anterior
Sensor de Velocidade Encoder (Acoplador Óptico):	1	Prof. Alfred Makoto
Bateria de Íons de Lítio (Li-ion 3.7V)	1	10,32

Módulo TP4056	1	6,92
Módulo Step-Up (Conversor Boost)	1	14,05
Chave On/Off Gangorra	1	Projeto anterior
Módulo Mini LED indicador de Bateria SMD	1	6,99
Bolinha de Aço Cromo	1	19,44
Impressão 3D	1	15,00
Total R\$72,72		

Fonte: autoria própria (18/05/2026)

Para a construção e o funcionamento deste módulo, foram selecionados os seguintes componentes:

ESP32 DevKit (Microcontrolador): Responsável por ler o sensor do jogo e transmitir o sinal de conclusão via Wi-Fi para a Caixa.

Sensor de Velocidade Encoder (Acoplador Óptico): Utilizado no final do percurso para detectar a bolinha de aço através da interrupção do feixe de luz na fenda do sensor.

Bateria de Íons de Lítio (Li-ion 3.7V): Fonte de alimentação portátil do módulo.

Módulo TP4056: Placa responsável pelo gerenciamento de carga e proteção da bateria Li-ion.

Módulo Step-Up (Conversor Boost): Responsável por elevar a tensão de 3.7V da bateria para os 5V estáveis exigidos pela ESP32.

Chave On/Off Gangorra: Interruptor mecânico para ligar e desligar o circuito, garantindo o controle de energia.

Módulo Mini LED indicador de Bateria SMD: Placa compacta de leitura visual rápida

que indica o nível de carga atual da bateria através de uma barra de LEDs.

Bolinha de Aço Cromo: Elemento físico que percorre o labirinto.

A bateria Li-ion é conectada diretamente ao módulo TP4056, que por sua vez está ligado em paralelo com o Step-Up. Para garantir o controle de energia de todo o sistema, uma chave On/Off foi instalada em série no fio positivo (VCC) que alimenta a entrada do circuito. A partir do Step-Up, a tensão elevada e regulada alimenta a ESP32, que fica responsável por fornecer a energia necessária para o funcionamento do sensor acoplador óptico, realizando a leitura contínua do estado do feixe infravermelho. Quando a bolinha passa pela fenda do sensor e interrompe esse feixe, a alteração no sinal digital faz a ESP32 disparar a rotina de destravamento em etapas da Maleta.

2.4.12 - Bateria (Tempo Estimado de Duração e Carregamento)

Tempo Estimado de Duração: Em testes de funcionamento contínuo, a bateria apresentou uma autonomia de 5 horas ininterruptas de uso até atingir a tensão limite de 3 Volts.

Tempo de Carregamento: O processo completo de recarga através do módulo TP4056 leva em torno de 2 horas e 30 minutos para o tipo de bateria deste circuito.

Nota de Segurança: Para preservar a vida útil e a durabilidade dos eletrodos da bateria Li-ion, deve-se evitar que a carga caia abaixo do limite mínimo de 3V. Além disso, é fundamental manter a chave liga/desliga (On/Off) do circuito na posição desligada durante todo o processo de carregamento.

2.4.13 - Esquemático

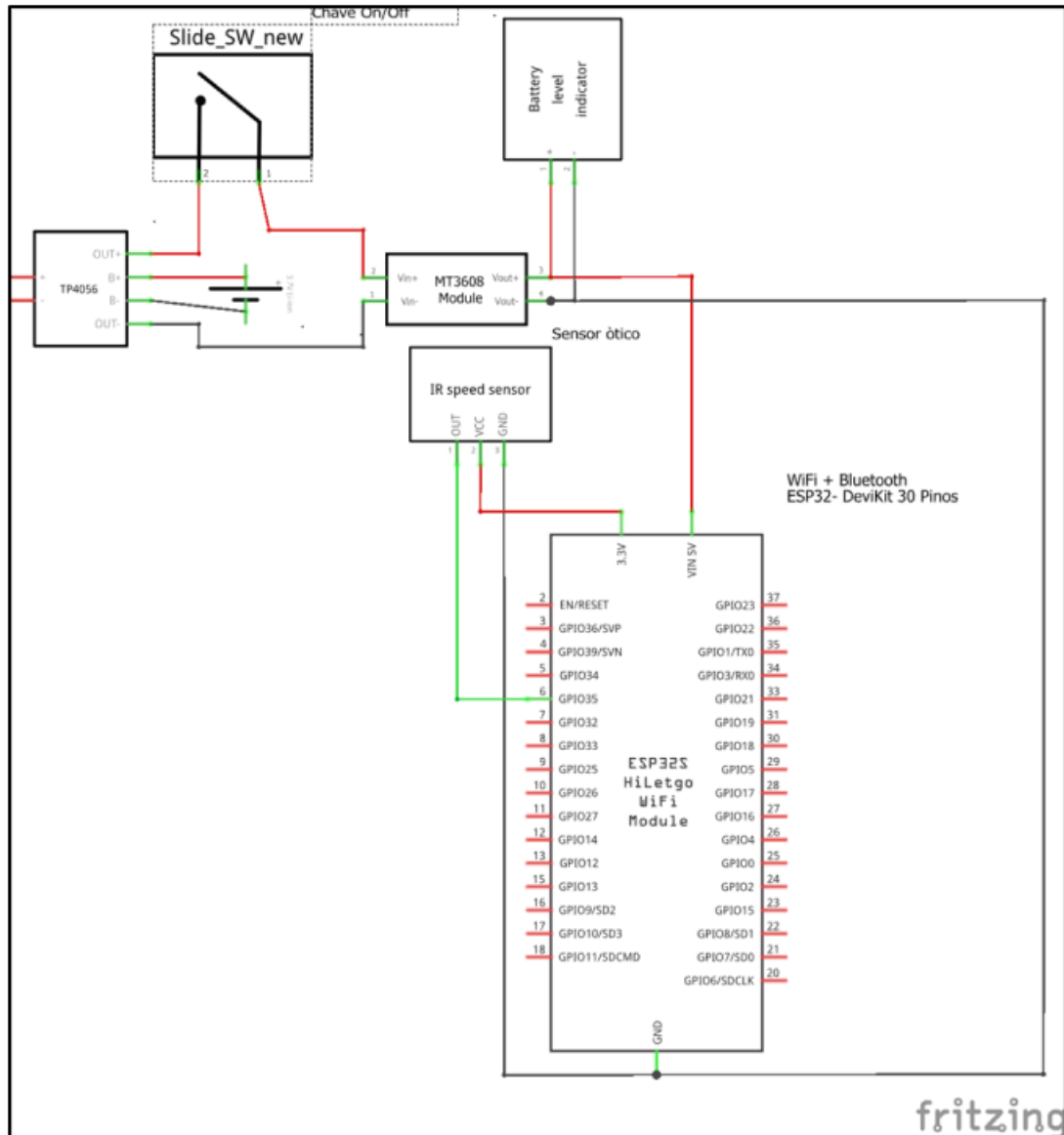
O circuito foi planejado para otimizar as conexões elétricas e a distribuição de energia de forma direta, Figura 25:

1. A Bateria Li-ion conecta-se aos pinos de entrada de bateria do TP4056.
2. A saída do TP4056 passa pela chave liga/desliga gangorra antes de chegar à entrada do módulo Step-Up.

3. A saída regulada de 5V do Step-Up é ligada ao pino VIN da ESP32.
4. O GND é compartilhado entre todos os módulos do circuito.

Figura do Esquemático

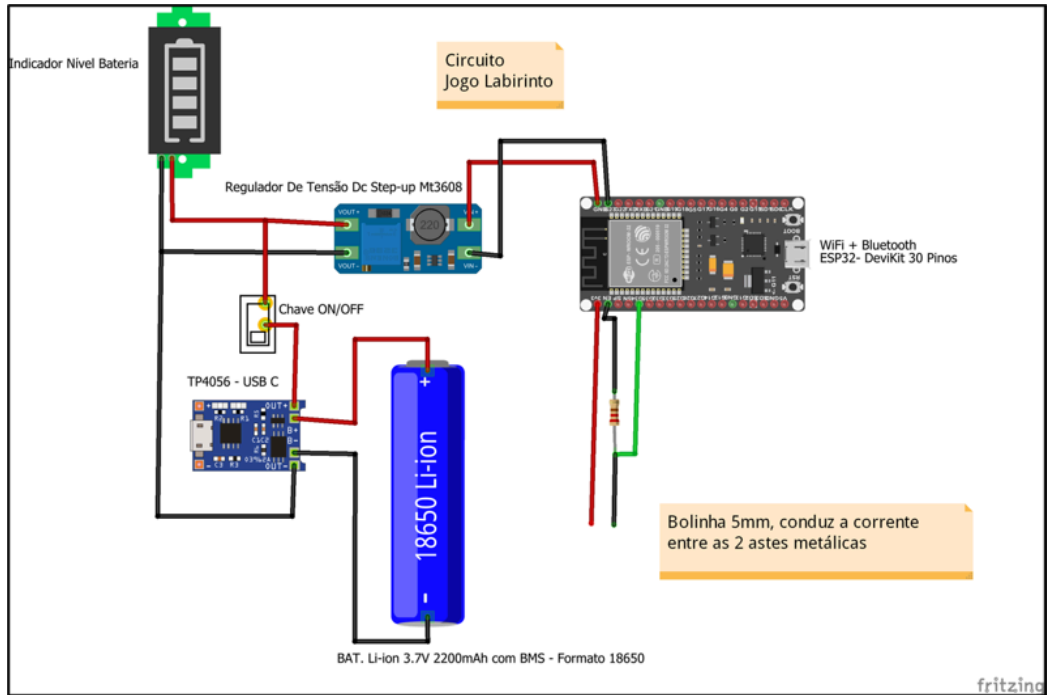
Figura 30 - Esquemático do Circuito



Fonte: autoria própria (18/05/2026)

2.4.14 - Diagrama Elétrico

Figura 31 - Diagrama Elétrico do Projeto



Fonte: autoria própria (18/05/2026)

2.4.15 - Estrutura (Contextualização do Antigo para o Novo)

A evolução da estrutura física focou na portabilidade e na melhoria da experiência do usuário, comparando a versão anterior com o modelo atual, Tabela 2.4B:

Tabela 2.4B- Contextualização

Característica	Versão Antiga	Nova Versão Atualizada
Dimensões	180 mm x 180 mm	90 mm x 90 mm x 50 mm
Sensor	Segue faixa - TCRT5000	Sensor ótico – LM393
Bateria	Li-Po 1500 mha	Li-ion 2200 mha

Carregador	Não existia	Módulo TP4056
------------	-------------	---------------

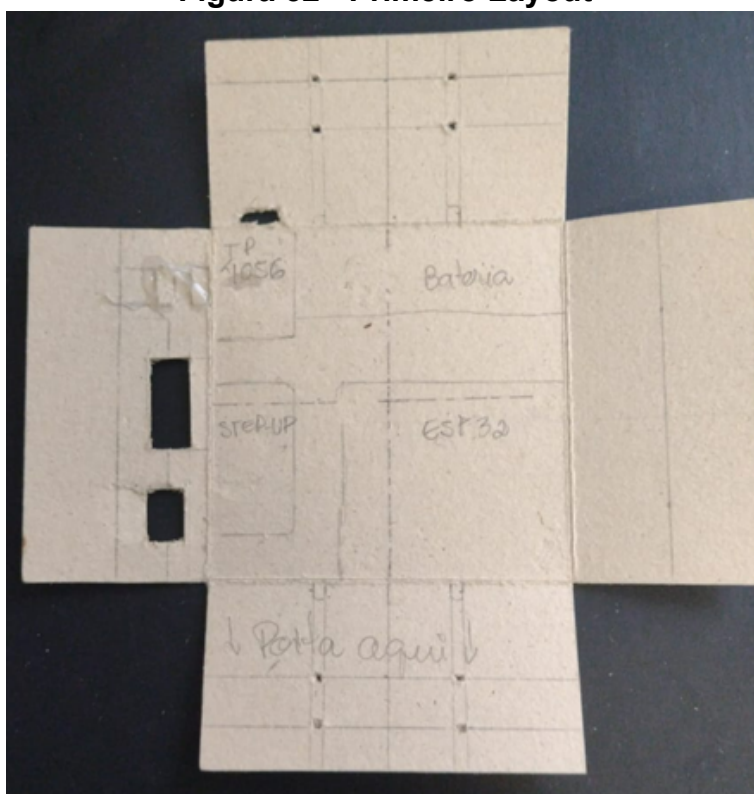
Fonte: autoria própria (18/05/2026)

Para desenvolver essa versão menor, partiu-se de uma comparação do tamanho dos componentes eletrônicos escolhidos (como base: ESP32, bateria, Step-Up, sensor encoder e TP4056).

O processo de design seguiu as seguintes etapas práticas:

Layout no Papel: Desenho inicial do tamanho dos componentes em uma folha para criar uma disposição que fizesse tudo funcionar, reduzindo o espaço total, Figura 27.

Figura 32 - Primeiro Layout

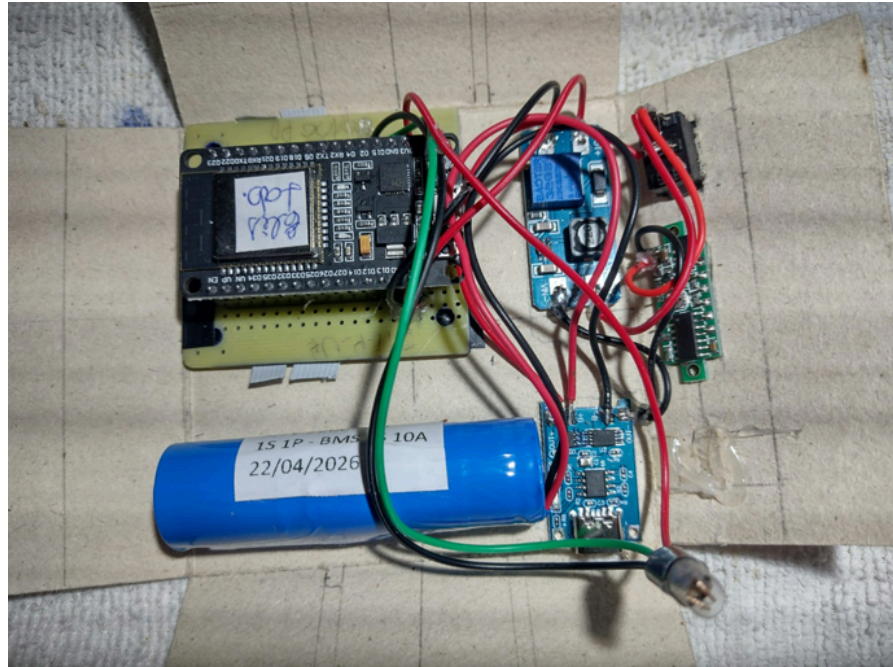


Fonte: autoria própria (18/05/2026)

Protótipo em Papelão: Montagem de um modelo físico em papelão para validar a disposição das peças e testar o envio do sinal para a caixa. Neste primeiro modelo foi removido o sensor segue faixa, no lugar, foi inserido um modelo de detector no estilo de botão em pulldown, somente com os fios cada um ligado a uma haste metálica dentro de um canudo coletando assim a bolinha que faria a ligação entre as duas

hastes, porém, houve um problema de bouncing elevado. Rejeitando então essa primeira tentativa. Segue imagem do modelo montado Figura 28.

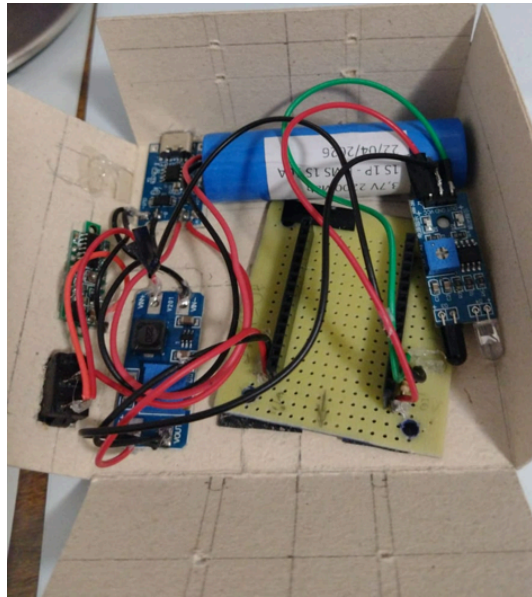
Figura 33 - Circuito da Primeiro Modelo em papelão



Fonte: autoria própria (18/05/2026)

Na segunda tentativa, retornou-se ao uso do sensor segue-faixa com o objetivo de detectar a bolinha de aço. Contudo, pelo fato de a bolinha possuir apenas 5 mm de diâmetro, o seu tamanho reduzido dificultou a leitura pelo sensor. Além disso, a geometria esférica da bolinha reduziu a superfície de reflexão de luz, o que comprometeu a precisão do componente e impediu que ele entregasse leituras corretas e estáveis, Figura 29.

Figura 34 - Circuito da Segundo Modelo em papelão



Fonte: autoria própria (18/05/2026)

Na terceira tentativa, foi implementado um sensor magnético, o qual funcionou perfeitamente na detecção. No entanto, essa abordagem tornou o retorno da bolinha ao fluxo do jogo impraticável, inviabilizando a dinâmica do circuito.

Na quarta tentativa, realizou-se o teste com modelos reduzidos de chaves fim de curso, incluindo a versão em SMD. Contudo, devido ao diâmetro de 5 mm, a bolinha de aço não possuía massa suficiente para exercer a força necessária e acionar o contato mecânico desses sensores.

Na quinta tentativa, utilizou-se o sensor óptico do tipo "encoder" baseado no comparador LM393. Esta solução apresentou um funcionamento excelente, mostrando-se ideal para detectar a passagem da bolinha de forma precisa e sem a necessidade de contato físico ou reflexão de luz. Figura 32.

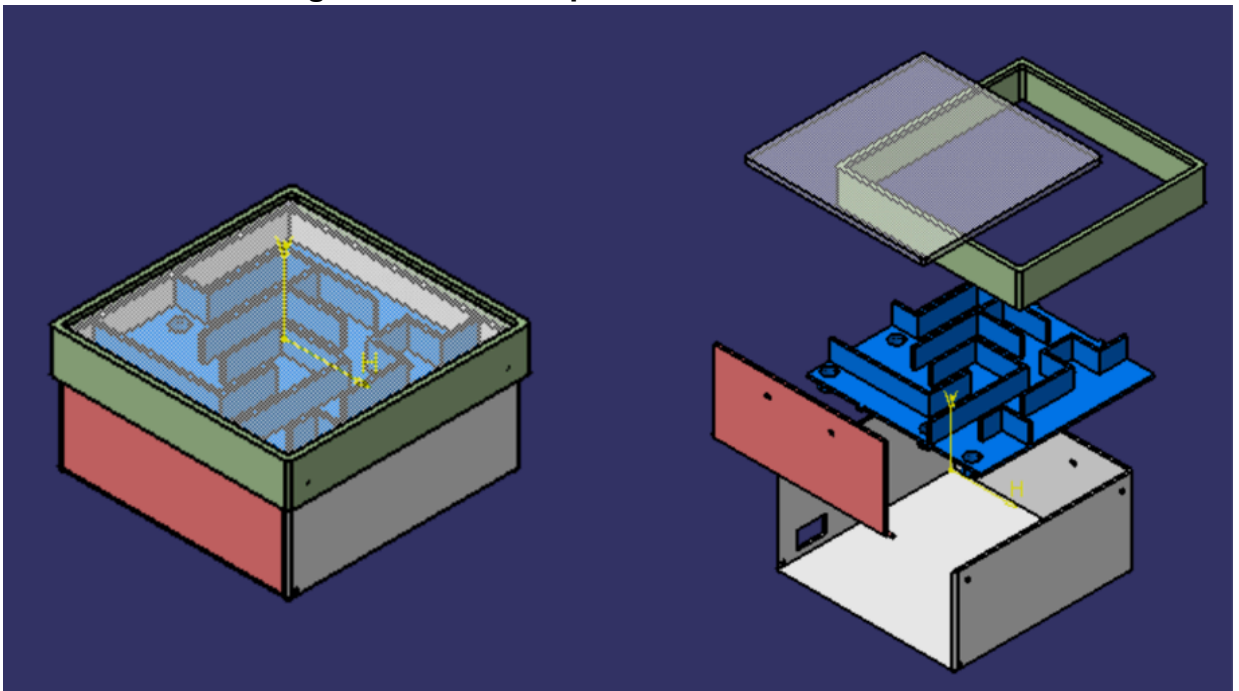
2.4.16 - Desenvolvimento do Invólucro em 3D

O invólucro do jogo foi desenvolvido para otimizar a montagem física e a organização dos componentes em formato modular. O desenho tridimensional foi estruturado no software Catia V5 e dividido em partes específicas: a base do invólucro; uma porta articulada para abertura e fechamento; e a tampa superior, projetada para ser integrada a um pedaço quadrado de acrílico. A estrutura interna do labirinto foi

planejada com paredes e buracos de fundo fechado para guiar e facilitar a coleta da bolinha. Apenas o buraco correspondente ao objetivo final foi mantido aberto, posicionado em contato direto com o sensor óptico LM393 para detectar a bolinha e enviar o sinal para a Maleta.

Durante a fase de modelagem no Catia V5, uma animação digital foi realizada para simular o encaixe das peças e verificar o funcionamento mecânico antes da fabricação, Figura 30.

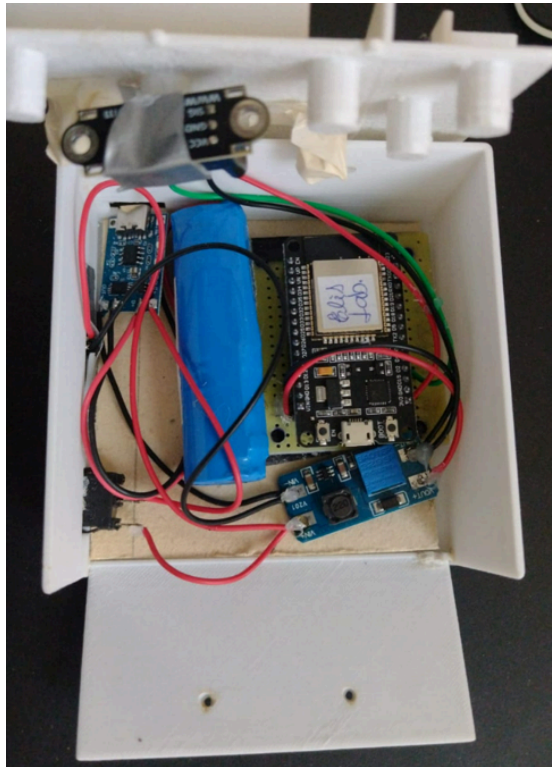
Figura 35 - Vista explodida do desenho CAD



Fonte: autoria própria (18/05/2026)

Para a execução da impressão 3D, os arquivos foram exportados no formato .STL e fatiados no software Bambu Studio, da Bambu Lab. Todas as peças foram impressas em filamento PLA, primeiramente, na cor branca, Figura 31.

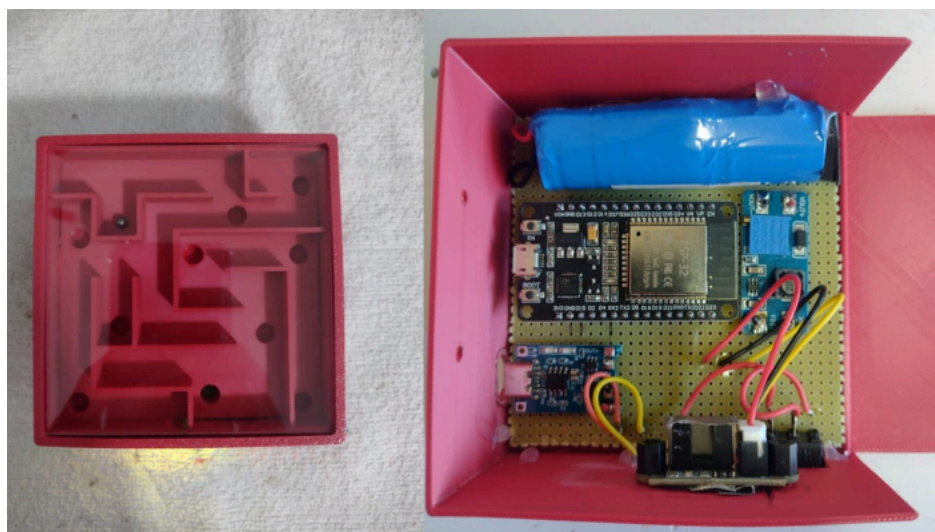
Figura 36 - Circuito do Modelo Impresso 3D



Fonte: autoria própria (18/05/2026)

Para a versão final do protótipo, foram executados pequenos ajustes geométricos visando garantir o perfeito encaixe e acoplamento de todos os componentes internos e estruturais. Após essas validações dimensionais, as peças definitivas foram impressas utilizando filamento PLA na cor vermelha, Figura 32.

Figura 37 - Modelo 3D Impresso e Montado



Fonte: autoria própria (18/05/2026)

2.4.17 - Conclusão

O desenvolvimento e a montagem do Jogo Labirinto permitiram alcançar os objetivos definidos para o projeto. A atualização da estrutura física trouxe uma redução significativa de tamanho em relação à versão anterior, diminuindo as dimensões de 180 mm para 90 mm, o que facilitou o encaixe do módulo na Maleta sem prejudicar a jogabilidade.

A sequência de testes e as cinco tentativas práticas foram essenciais para definir o melhor sistema de leitura. A escolha do sensor óptico "encoder" LM393 resolveu os problemas de mau contato, falta de peso da bolinha de 5 mm e falhas de reflexão que ocorreram nos sensores testados anteriormente. Na parte de energia, a inclusão da bateria de 2200 mAh junto ao módulo de carga TP4056 e ao indicador de LED SMD garantiu o funcionamento seguro do circuito, alcançando uma autonomia de 5 horas de uso contínuo.

Por fim, o trabalho de desenho no Catia V5 combinado com a impressão 3D em PLA resultou em um acabamento sob medida e resistente. Com a comunicação sem fio configurada via Wi-Fi entre as placas ESP32, o módulo cumpre sua função de forma integrada, enviando o sinal de término para o acionamento da Íris da Maleta.

3. CONCLUSÃO

O projeto MEDLock atingiu seus objetivos principais, funcionando de acordo com o planejado. A gamificação mostrou-se uma ferramenta eficaz para promover engajamento e raciocínio lógico, criando uma experiência interativa inspirada no conceito de escape room. A caixa principal, com seu sistema de trava em íris controlada por ESP 32, respondeu corretamente aos sinais enviados via wifi localhost pelos jogos concluídos. Os jogos foram integrados com sucesso e cumpriram sua função de destravar as respectivas etapas da caixa.

A divisão de tarefas entre os membros da equipe e o esforço coletivo na documentação e integração foram fundamentais para o desenvolvimento do projeto, que empregou tecnologias como o ESP 32 e circuitos de proteção de forma eficiente. A experiência adquirida reforça o potencial da integração entre hardware, software e design para criar soluções educativas e interativas.

REFERÊNCIAS

Alvaro, Julie. Ácido Polilático (PLA). QUIÍMICA.com.br. Disponível em: <https://www.quimica.com.br/#:~:text=Propriedades%20do%20%C3%A1cido%20polil%C3%A1tico,com%%2020variados%20graus%20de%20flexibilidade> . Acesso em: 03 Novembro 2025.

MIT APP INVENTOR. Disponível em <https://appinventor.mit.edu/>

Extensão taifun wifi. Disponível em <https://puravidaapps.com/wifi.php>

Extensão Tetris para APP Inventor. Disponível em <https://community.appinventor.mit.edu/t/tetris-recreation-of-1980s-retro-game/51075>

Tutorial de como criar um jogo tetris no MIT APP Inventor. Disponível em <https://www.youtube.com/watch?v=TxEd2qrdPBc>

APÊNDICE A - Programação completa do Painel da Íris.

```
1 //Programação final
2 #include <WiFi.h>
3 #include <WebServer.h>
4 #include <ESP32Servo.h>
5 #include <TM1637Display.h>
6
7 Servo servo;
8
9 // Display TM1637
10 const int CLK = 5; // G5
11 const int DIO = 4; // G4
12 TM1637Display display(CLK, DIO);
13
14 // LEDs AZUIS
15 const int ledLabB = 16;
16 const int ledTetB = 17;
17 const int ledMemB = 18;
18
19 const int botao = 23;
20 int etapaAtual = 0;
21 int posicaoInicial = 180;
22 const int ajuste = 33;
23
24 // Variáveis de controle dos jogos
25 int labirinto = 0;
26 int tetris = 0;
27 int memoria = 0;
28
29 // Flags para garantir que cada etapa só seja acionada 1 vez
30 bool etapaLabExecutada = false;
31 bool etapaTetExecutada = false;
32 bool etapaMemExecutada = false;
33
34 const int servoPin = 25;
35
36 // WiFi AP
37 const char* ssid = "INNOWAVE";
38 const char* password = "SSID1234";
39
40 WebServer server(80);
41
42 // Controle do tempo (Timer Regressivo)
43 const unsigned long TEMPO_LIMITE_MS = 540000; // 9 minutos em milissegundos
44 unsigned long tempoInicio = 0;
45 unsigned long tempoFinalizado = 0; // Armazena o tempo decorrido até a vitória
46 bool cronometroAtivo = true;
```

```

47
48 void setup() {
49     Serial.begin(115200);
50
51     display.setBrightness(7);
52     display.showNumberDecEx(0, 0b01000000, true); // 00:00
53
54     WiFi.softAP(ssid, password);
55     Serial.print("IP do servidor: ");
56     Serial.println(WiFi.softAPIP());
57
58     server.on("/", handleRoot);
59     server.on("/ledLab/on", ledPinLabOn);
60     server.on("/ledTet/on", ledPinTetOn);
61     server.on("/ledMem/on", ledPinMemOn);
62
63     server.begin();
64
65     servo.setPeriodHertz(50);
66     servo.attach(servoPin, 500, 2400);
67
68     pinMode(botao, INPUT_PULLUP);
69
70     pinMode(ledLabB, OUTPUT);
71     pinMode(ledTetB, OUTPUT);
72     pinMode(ledMemB, OUTPUT);
73
74     fecharTudo();
75 }
76
77 void loop() {
78     server.handleClient();
79
80     atualizarDisplayTempo();
81
82     if (digitalRead(botao) == LOW) {
83         delay(10);
84
85         if (digitalRead(botao) == LOW) {
86             fecharTudo();
87
88             while (digitalRead(botao) == LOW) {
89                 delay(10);
90             }
91         }
92     }
93 }

```

```

94
95 void atualizarDisplayTempo() {
96     unsigned long tempoDecorrido;
97
98     if (cronometroAtivo) {
99         tempoDecorrido = millis() - tempoInicio;
100     } else {
101         tempoDecorrido = tempoFinalizado;
102     }
103
104     long tempoRestanteMs = TEMPO_LIMITE_MS - tempoDecorrido;
105
106     // Se o tempo esgotar
107     if (tempoRestanteMs <= 0) {
108         tempoRestanteMs = 0;
109
110         // Se o tempo acabou e o jogo ainda estava rolando, força a parada
111         if (cronometroAtivo) {
112             cronometroAtivo = false;
113             tempoFinalizado = TEMPO_LIMITE_MS; // Trava o tempo decorrido no limite máximo
114         }
115     }
116
117     int segundosTotais = tempoRestanteMs / 1000;
118     int minutos = segundosTotais / 60;
119     int segundos = segundosTotais % 60;
120
121     int valorDisplay = minutos * 100 + segundos;
122
123     display.showNumberDecEx(valorDisplay, 0b01000000, true);
124 }
125
126 void handleRoot() {
127     String html = "<!DOCTYPE html><html>";
128     html += "<head><meta name='viewport' content='width=device-width, initial-scale=1'>";
129     html += "<style>html{font-family:Helvetica;text-align:center;}";
130     html += ".button{background:#4CAF50;color:white;padding:16px 40px;font-size:20px;border:none;}";
131     html += "</style></head>";
132
133     html += "<body>";
134     html += "<h1>INNOWAVE MED LOCK</h1>";
135     html += "<h2>Destrave a Caixa</h2>";
136
137     html += "<p><a href='/ledLab/on'><button class='button'>ETAPA 1</button></a></p>";
138     html += "<p><a href='/ledTet/on'><button class='button'>ETAPA 2</button></a></p>";
139     html += "<p><a href='/ledMem/on'><button class='button'>ETAPA 3</button></a></p>";
140

```

```

140
141     html += "</body></html>";
142
143     server.send(200, "text/html", html);
144 }
145
146 void ledPinLabOn() {
147     labirinto = labirinto + 1;
148     abrirEtapas();
149     server.setHeader("Location", "/");
150     server.send(303);
151 }
152
153 void ledPinTetOn() {
154     tetris = tetris + 1;
155     abrirEtapas();
156     server.setHeader("Location", "/");
157     server.send(303);
158 }
159
160 void ledPinMemOn() {
161     memoria = memoria + 1;
162     abrirEtapas();
163     server.setHeader("Location", "/");
164     server.send(303);
165 }
166
167 void fecharTudo() {
168     servo.write(180);
169
170     digitalWrite(ledLabB, LOW);
171     digitalWrite(ledTetB, LOW);
172     digitalWrite(ledMemB, LOW);
173
174     etapaAtual = 0;
175     posicaoInicial = 180;
176
177     labirinto = 0;
178     tetris = 0;
179     memoria = 0;
180
181     etapaLabExecutada = false;
182     etapaTetExecutada = false;
183     etapaMemExecutada = false;
184
185     // Reseta e reativa o timer regressivo para começar de 9:00 de novo
186     tempoInicio = millis();

```

```

186     tempoInicio = millis();
187     tempoFinalizado = 0;
188     cronometroAtivo = true;
189
190     display.showNumberDecEx(900, 0b01000000, true); // Inicia exibindo 09:00
191 }
192
193 void verificarFimDeJogo() {
194     if (etapaLabExecutada && etapaTetExecutada && etapaMemExecutada) {
195         if (cronometroAtivo) {
196             // Salva exatamente o tempo decorrido até o momento da vitória
197             tempoFinalizado = millis() - tempoInicio;
198             cronometroAtivo = false;           // Congela o timer regressivo
199         }
200     }
201 }
202
203 void abrirEtapas() {
204     // SEGURANÇA: Se o tempo chegou a zero (tempoFinalizado atingiu o limite),
205     // impede qualquer ação no servo ou nos LEDs. Bloqueia tudo!
206     if (tempoFinalizado >= TEMPO_LIMITE_MS) {
207         return; // Sai da função imediatamente sem fazer nada
208     }
209
210     // Etapa Labyrinth
211     if (labirinto >= 1 && !etapaLabExecutada) {
212         if (posicaoInicial > 81) {
213             posicaoInicial = posicaoInicial - ajuste;
214             servo.write(posicaoInicial);
215         }
216         digitalWrite(ledLabB, HIGH);
217         etapaLabExecutada = true;
218     }
219
220     // Etapa Tetris
221     if (tetris >= 1 && !etapaTetExecutada) {
222         if (posicaoInicial > 81) {
223             posicaoInicial = posicaoInicial - ajuste;
224             servo.write(posicaoInicial);
225         }
226         digitalWrite(ledTetB, HIGH);
227         etapaTetExecutada = true;
228     }
229
230     // Etapa Memória
231     if (memoria >= 1 && !etapaMemExecutada) {
232         if (posicaoInicial > 81) {

```